DEPARTMENT OF ELECTRONIC & ELECTRICAL ENGINEERING

University of Strathclyde Engineering

"I hereby declare that this work has not been submitted for any other degree/course at this University or any other institution and that, except where reference is made to the work of other authors, the material presented is original and entirely the result of my own work at the University of Strathclyde under the supervision of Dr Louise H Crockett."

SDR Application on Enterprise RFSoC Platform

Fraser O'Regan 202115033

Supervisor: Dr Louise H Crockett

April 2025

Acknowledgements

Firstly, I would like to thank Dr Louise H Crockett from the department of Electronic and Electrical Engineering at the University of Strathclyde. Your support and guidance throughout the duration of this project has been invaluable and I have thoroughly enjoyed working alongside you.

Secondly, I would like to thank both my family and girlfriend. This project would not be the same without your support, and I hope you're all FPGA experts after listening to me talk about them for the last six months. I would also like to thank my friends at the university for their continued support throughout the past 4 years.

Finally, I would like to thank BittWare and Strath-SDR for providing me with both technical support and hardware access throughout this project.

Abstract

Software-defined radio (SDR) has become the leading approach for developing, testing, and deploying modern radio systems. The AMD RFSoC is a specialised RF signal processing device that combines the hardware programmability and parallel processing capabilities of an FPGA with the software flexibility of a processor. As a state-of-the-art platform for SDR, the RFSoC provides comprehensive RF signal processing capabilities within a single device. This project focuses on developing an SDR system for an enterprise-level device, the BittWare RFX-8440. The system was designed using Vitis Model Composer to generate custom blocks for use within Vivado's IP Integrator. The custom IP was deployed on both the RFX and the RFSoC 2x2, allowing project development to continue even when access to the RFX was limited. The designs were tested and validated in both simulations and on hardware, using tools such as PYNQ and PetaLinux to interface between software and hardware. The system was deigned to produce and transmit an Amplitude Modulated (AM) waveform across an ideal channel using a loopback cable, where the signal was then recovered on the receiver side.

Contents

Nomenclature

Please see below all relevant nomenclature as seen in **alphabetical** order.

Abbreviations

ADC	Analogue to Digital Converter
AI	Artificial Intelligence
AM	Amplitude Modulation
AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
DAC	Digital to Analogue Converter
DMA	Direct Memory Access
DSP	Digital Signal Processing
FFT	Fast Fourier Transform
FIR	Finite Impulse Response (Filter)
FPGA	Field Programmable Gate Array
FT	Fourier Transform
GUI	Graphical User Interface
HDL	Hardware Description Language
HLS	High Level Synthesis
IC	Integrated Circuit
ILA	Integrated Logic Analyser
IP	Intellectual Property
IPI	IP Integrator
JTAG	Joint Test Action Group
LUT	Look Up Table
MUX	Multiplexer
NCO	Numerically Controlled Oscillator
OFDM	Orthogonal Frequency Division Multiplexing
OOT	Out Of Tree
OS	Operating System
PCIe	Peripheral Component Interconnect Express
PL	Programmable Logic
PS	Processing System
PLL	Phase Locked Loop
QPSK	Quadrature Phase Shift Keying
QSFP	Quad Small Form-factor Pluggable

Abbreviations (Cont.)

QSPI	Quad Serial Peripheral Interface
RF	Radio Frequency
RFDC	Radio Frequency Data Converter
RFSoC	Radio Frequency System-on-Chip
ROM	Read Only Memory
RTL	Register Transfer Level
SDK	Software Development Kit
SDR	Software-Defined Radio
SD − FEC	Soft Decision Forward Error Correction
SFDR	Spurious Free Dynamic Range
SoC	System-on-chip
Tcl	Tool command language
USRP	Universal Software Radio Peripheral
XDC	Xilinx Design Constraints (.xdc)
XSA	Xilinx Support Archive (.xsa)

Symbols

Please see all relevant mathematical symbols below as seen in **chronological** order throughout this report.

$X(j\omega)$	Fourier Transform of signal $x(t)$
e	Euler's Number
j	The Imaginary Number ($\sqrt{-1}$)
ω	Angular Frequency
f_s	Sampling Frequency
f_{max}	Maximum Frequency Component of Signal
μ	Step Size
N	Depth of LUT in NCO
f_d	Desired Frequency from NCO
n	Number of Bits

1 Introduction

Modern advancements in Digital Signal Processing (DSP), along with Artificial Intelligence (AI), increased processing power, and other technological developments, have culminated in the birth of the Software-Defined Radio (SDR). SDR utilises the flexibility and configurability of an analogue Radio Frequency (RF) front end [1], along with the *on-the-go* adaptable nature of software to create a fully programmable and customisable communications system. SDR is traditionally defined as any radio system in which one or more elements is software-defined. Nowadays, radios can be designed almost entirely in software, allowing for even more flexibility, with less reliance on bulky hardware. SDR is the backbone of modern development into communication systems, providing cost-effective solutions for exploring 5G and beyond. There are many supporting frameworks for SDR development, but GNU Radio is seen as one of the most popular software for SDR design. GNU Radio is an open source project built primarily in C++ and Python [2]. GNU Radio also has its own Graphical User Interface (GUI), the GNU Radio companion, an intuitive block diagram modelling tool which aids in the development of these systems without the need to extensively write and debug software. The software is configurable with a number of low end hardware devices, however, it technically possesses the ability to be ported to any custom hardware that the user may need through an 'out of tree module' (OOT).

The primary aim of this project is to implement an SDR system on a novel piece of hardware, the BittWare RFX-8440, marking a new endeavour for BittWare, as this kind of technology has not yet been applied to their custom hardware devices. The objectives include potentially exploring the integration of GNU Radio with the BittWare platform, leveraging its high-performance capabilities to develop a functional SDR system, and evaluating its performance against traditional setups. Additionally, a key aim of the project is to gain familiarity with the architecture of the AMD RFSoC, its supporting toolchains, and design flows. This knowledge will lay a solid foundation for progressing toward the design and development of custom SDR systems. Additionally, with the system up and running, some extra goals have been set to provide further challenges in validating the systems design. These include the transmission of radio waves over the air, and the potential to receive and transmit some "real data", i.e. text, images, audio, etc. Of course, ensuring proper care and licensing issues have been dealt with before engaging in such activities is imperative when working with wireless transmission.

2 Background & Literature Review

2.1 RFSoC Architecture

This project will focus on the implementation of the aforementioned software and design techniques onto the Zynq UltraScale+ RFSoC (which from now on will be referred to as the RFSoC) by AMD [3]. The RFSoC is an all configurable system-on-chip (SoC) device targeted towards RF applications [4]. A system-on-chip is an integrated circuit (IC) that features almost all high level functions needed for computation into a single chip, as opposed to the traditional method of having these different components mounted to the motherboard. This allows for a far smaller and more portable package, making it ideal for applications such as communications, mobile devices, and many more. The RFSoCs architecture features two main sections, the Processing System (PS), and the Programmable Logic (PL). The AMD RFSoC utilises the software programmability of processors, in the form of either two or four ARM cores (depending on the generation), along with the hardware programmability of a Field Programmable Gate Array (FPGA). The PL features a number of different "hardened" blocks, aptly named due to the fact that they have their own dedicated space on the silicon and cannot be reconfigured in either software or hardware. The different hardened blocks can be seen below in Figure 1.

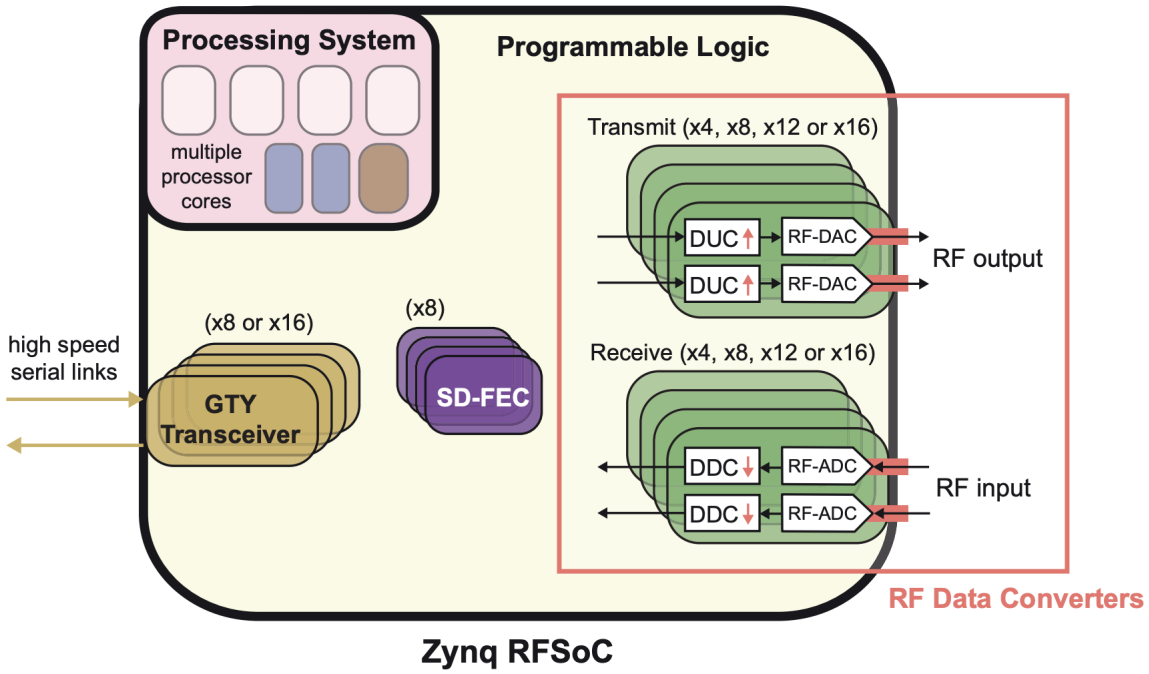

Figure 1: High Level Overview of RFSoC Architecture [4]

From left to right, the hardened blocks shown in Figure 1 are: Gigabit (GTY) transceivers, a high speed serial data link needed for communication to a core network when working with modern communication standards. Additionally, Soft Decision Forward Error Correction (SD-FEC) blocks feature on the chip, since almost all forms of wireless communications require, at some stage, an error correction phase which

mitigates against the different errors procured in the wireless channel. Finally, the RFSoC has dedicated RF Data Converter (RFDC) blocks. These blocks can be configured as either Digital to Analogue Converters (DACs), for data transmission, or Analogue to Digital Converters (ADCs), which are used to convert real data to the digital domain, where the signal can be analysed using numerous DSP techniques.

The Advanced eXtensible Interface (AXI) is a communication protocol defined by ARM as part of the Advanced Microcontroller Bus Architecture (AMBA) standard [5]. The AXI4 standard has three different forms: AXI4-Lite, for simple, low-throughput communications, for example, driving an enable register pin; AXI4-Stream, for high-speed streaming data; and full AXI4, which is used for high-performance applications. Typically, embedded systems, such as FPGAs, operate with a master-slave topology between different devices [6]. The master device can connect to multiple slave devices through a communication protocol, and slave devices can be controlled by multiple masters, executing commands only when driven by a master device. AXI follows this same principle, with read and write channels transferring data between the master and slave components in a hardware design.

AXI4 uses handshake operations, where a handshake is successful when both the valid and ready signals are high on the same clock cycle [6]. An example of an AXI4 handshake can be seen below in Figure 2.

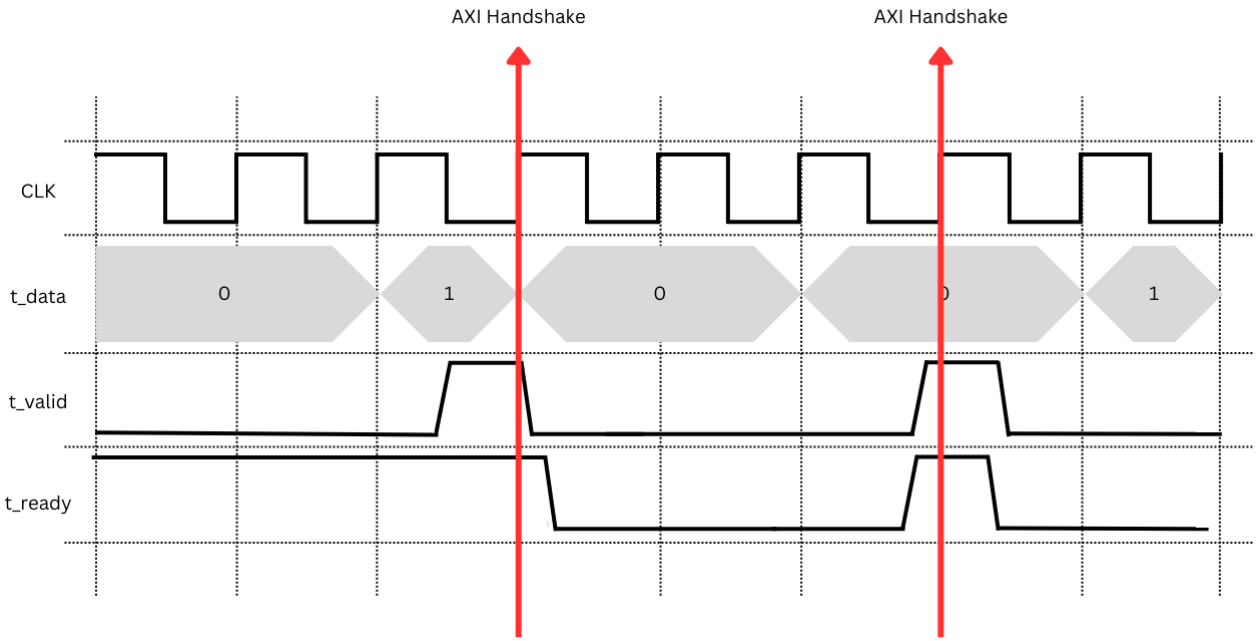

Figure 2: Example Graph of AXI4 Handshakes

AXI4-Lite has both a read and write transaction, with the former using the **read address** and **read data** channels, and the latter using the **write address**, **write data**, and **write response** channels [6]. To summarise an AXI4-Lite handshake, the read transaction is performed when the master sends an address on the **read address** channel, after which the **valid** and **ready** signals are asserted, performing the handshake. This is then followed by the read operation where the slave sends the data at the requested memory address using the **read data** channel. To perform write operations, the master sends an address value using the

write address channel, which performs a basic handshake. The master then sends the data that is to be written into the memory address to the slave, this uses the **write data** channel. Finally, the slave responds using the **write response** channel, which indicates to the master whether the write operation was successful or not. There is vast support for AXI4-Lite on AMD devices, especially using Vitis Model Composer, where inputs to designs (Gateway In)(s) can be implemented as an AXI4-Lite register, which allows for simple control once the designs are generated into Intellectual Property (IP) cores in Vivado.

AXI4-Stream does not need a memory address channel since it does not move data between different memory locations, but rather moves data between different points on the FPGA. AXI4-Stream uses nine signals, but the most relevant can be seen below in Table 1.

Signal	Function
TREADY	Indicates that the slave device is ready to accept data in the current clock cycle
TVALID	Indicates the master is sending valid data. A transfer occurs if TREADY and TVALID are high.
TDATA	Provides the data that is transferred in an AXI4-Stream transaction
TLAST	Indicates the last data element in TDATA

Table 1: AXI4-Stream Signal Definitions

All AXI4-Stream signals propogate from master to slave, except for TREADY, which travels in the opposite direction. To implement AXI4-Stream in a design, there is a naming convention that must be followed, that is:

$$< Role > \_ < ClassName > \_ < SignalName >$$

For example, to implement a stream of data on the master side, the signal would be named:

$$m\_axis\_tdata$$

The AXI4 protocol plays a fundamental role in modern embedded systems, particularly in AMD SoC devices. In this project, AXI4-Stream is essential for efficiently transferring large volumes of data between multiple IP cores within the FPGA, ensuring high-performance communication and system integration.

2.2 Supporting Toolchains and Software

The PYNQ project by AMD [7] is another supporting framework that focuses on SoC devices. PYNQ utilises Python application programming interfaces (APIs) to streamline the development of embedded systems on AMD chips. PYNQ itself is a small linux based boot image that runs on the chip, with different commands to control the hardware; however, PYNQ also utilises Jupyter notebooks, in which python code can be ran in different cells, along with accompanying markdown code for readability. Hardware Description Language (HDL) “bitstreams” can be accessed in the notebook as “overlays”. The overlays must be

used in the project by exporting the Vivado block design, and locating the ***.hwh*** and ***.bit*** files. From that point, they can be implemented as needed by an user and reused indefinitely, unless modifications to the overlay are required. There are many methods to design a custom overlay to be used in PYNQ; however, the most common ways include writing custom Register Transfer Level (RTL) code using VHDL or Verilog; Writing C/C++ code to be synthesised into HDL code through AMDs Vitis High Level Synthesis (HLS) tool [8]; or through Vitis Model Composer, a block based modelling system that uses MATLAB and Simulink to generate HDL code [9].

This project focuses on the BittWare RFX-8440 (referred to as the RFX from now on) L band transceiver card [10]. A high performance hardware device powered by a Gen3 RFSoC, with up to eight RFDCs, featuring Multi gigahertz bandwidth for any SDR applications necessary. For communication with a host PC, the RFX uses Peripheral Component Interconnect Express (PCIe) connection, a high speed serial bus used for connecting peripheral components to a computers motherboard. Figure 3 shows the RFX-8440 from BittWare.

Figure 3: BittWare RFX-8440 Transceiver Card

Traditionally, PYNQ is only natively available on certain development boards created by AMD such as the RFSoC 2x2, 4x2, or the PYNQ Z-2 board. For PS-PL communication, the RFX uses PetaLinux, a Linux Software Development Kit (SDK) used for embedded Xilinx (AMD) processing systems. PetaLinux is a powerful tool for embedded system design, however, it is less user friendly than PYNQ due to the user needing a proprietary knowledge of the Linux kernel. PetaLinux is a small Linux based operating system which targets a specific hardware image using an exported Xilinx Support Archive (.xsa) file, and runs entirely on the PS.

Linux based operating systems (OS) are uncommon in most home laptops/PCs but are extremely common in industry applications. Linux is almost guaranteed to be the operating system of choice for data centres, R&D labs, servers, and even more. Linux is extremely important to the infrastructure of communication

systems, as all network protocol servers will run some sort of Linux distribution. Linux is popular due to its open-source nature, meaning that anyone has access to the source code, giving them the ability to customise the operating system and create specific distributions.

GNU Radio has many tutorials to get someone up to speed with the software, as well as having a recommended reading section for general SDR and communication techniques. These tutorials range from simple Amplitude Modulation (AM), to Orthogonal Frequency Division Multiplexing (OFDM), a far more advanced communication technique, where a signal is split up into multiple narrowband carrier frequencies rather than taking up lots of bandwidth with one carrier frequency [11]. These tutorials are an extremely useful learning tool, as they provide an interactive experience where someone can build different types of communication systems along with learning about them.

GNU Radio has been around for over 20 years, and with the ever growing application and need for SDR systems to be used in everyday environments, GNU Radio has served as an incredibly useful tool. Whether someone is new to SDR or is a veteran, GNU Radio provides a fantastic platform for developing these systems. As mentioned earlier, GNU Radio provides something called an OOT module, this is essentially a custom block that is either written in C++ (for more computationally intensive DSP) or Python (for simpler tasks). These OOT modules are aptly named since they are not part of GNU Radios 'source tree' [10], or more simply, are not native to the software. There are a number of heavily supported hardware that GNU Radio mentions on their website. This hardware is usually relatively inexpensive which makes it great for beginners, however, the RFX is not one of these boards. GNU Radio has something called the 'Catch-All Clause' which states that any hardware device that can be accessed by an operating system can support GNU Radio, usually by writing custom device drivers in the form of sinks or sources. Since this project focuses mostly on the deployment of an SDR system, there is a workaround for writing custom drivers for the RFX.

There has been a significant amount of work already accomplished within the GNU Radio community, ranging from simple transceivers to more complex designs, such as cognitive radio applications [12]. However, many of these designs have been implemented on the Universal Software Radio Peripheral (USRP), a dedicated SDR device that offers full support for GNU Radio. Since the RFX does not provide this level of native support, an alternative approach must be developed.

There is a novel SDR design that featuring both the RFSoC and GNU Radio, as referenced in [13]. This design features GNU Radio running on a host PC whilst an RFSoC 4x2 development board is acting as a receiver for a wideband signals. The 4x2 sends packeted data from the board to the host PC where GNU Radio displays these signals using the gr-fosphor package, a GPU accelerated spectrum visualiser. The signals are sent to the host PC from the 4x2 via the Quad Small Form-factor Pluggable (QSFP) transceiver, commonly used for high-speed data transmission.

The RFX features support for QSFP communication, allowing for a number of possibilities where packeted data can be offloaded to a different system, not necessarily running GNU Radio, but any other software performing DSP. This aforementioned implementation of GNU Radio saves significant design effort, as

there is no need for custom drivers to be written to support the enterprise hardware.

2.3 Important DSP Fundamentals

Throughout this project, a number of DSP and communication theories were used to design, assess, and confirm the operation of different systems within this project. This section will cover a number of basic DSP and communication fundamentals to allow the reader to have a better understanding of the different concepts being discussed throughout this project. Arguably the most important concept in DSP (and signal processing in general) is the Fourier Transform (FT). The FT is a highly complex concept with many different applications in different fields of engineering and science, so this report will not dive particularly deep into the derivation of how the FT came to be, but will focus more on its definition, and how it is used in this case. The FT stems from the Fourier series, discovered by Joseph Fourier, which essentially states that any periodic signal can be decomposed into a sum of different harmonically related sine and cosine components [4]. In the context of signal processing, the FT takes any signal, which is assumed to be periodic (if not - it is assumed the signal is periodic at $t = \infty$), and produces a number of coefficients that represent a magnitude and phase of the signal over time. These coefficients can then be plotted to produce a waveform which can tell an engineer about the frequency properties of a signal. The FT is defined in Equation 1 [14].

$$X(j\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}dt \tag{1}$$

Taking the FT of a waveform can be thought of as transforming the signal from the time domain, to the frequency domain. To illustrate this, a time domain and its frequency domain counterpart were plotted in MATLAB. This is shown in Figure 4.

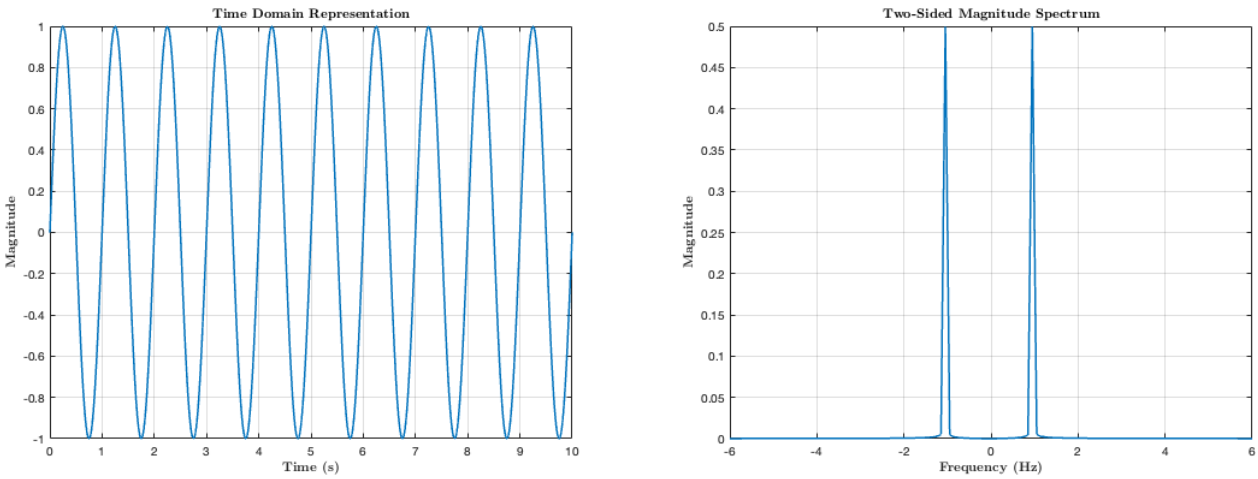

Figure 4: Time and Frequency Domain Representations of Sinusoid at 1 Hz

The FT of the signal shows a peak at 1 Hz, which is the exact frequency of the signal present. However,

in more real world examples, the FT of a signal can show multiple peaks at different frequencies as signals can have many different frequency components. In most cases, the FT is not directly computed, as it is quite inefficient to calculate as it takes n^2 operations. More commonly, a DSP algorithm called the Fast Fourier Transform (FFT) of a signal is calculated as it is far more computationally inexpensive, allowing for more complex signals to be analysed. Also note that this is just a light explanation of the FT and there are far more details that have been omitted for clarity.

In addition to the FT, a fundamental theory of DSP is the Nyquist sampling theory. This theory is used to describe how often a signal should be sampled in order to fully represent itself in both the time and frequency domain. Nyquist's sampling theory is expressed in Equation 2.

$$f_s \geq 2f_{max} \tag{2}$$

Where f_{max} denotes the highest frequency component present in the sampled signal. A high sampling rate is essential to prevent aliasing - a phenomenon that occurs when a signal is sampled below the Nyquist rate, causing higher frequency components to be "folded" into the first Nyquist zone [4]. Nyquist zones are defined as frequency intervals of width $\frac{f_s}{2}$, with the first zone spanning from 0 Hz to $\frac{f_s}{2}$ Hz. Typically, signals are sampled higher than the Nyquist rate to ensure aliasing does not occur, as well as providing a clear representation of the signal for transmission.

The digital logic in this design will realise an AM transmitter (T_X) and receiver (R_X). A high level overview of this system can be seen below in Figure 5.

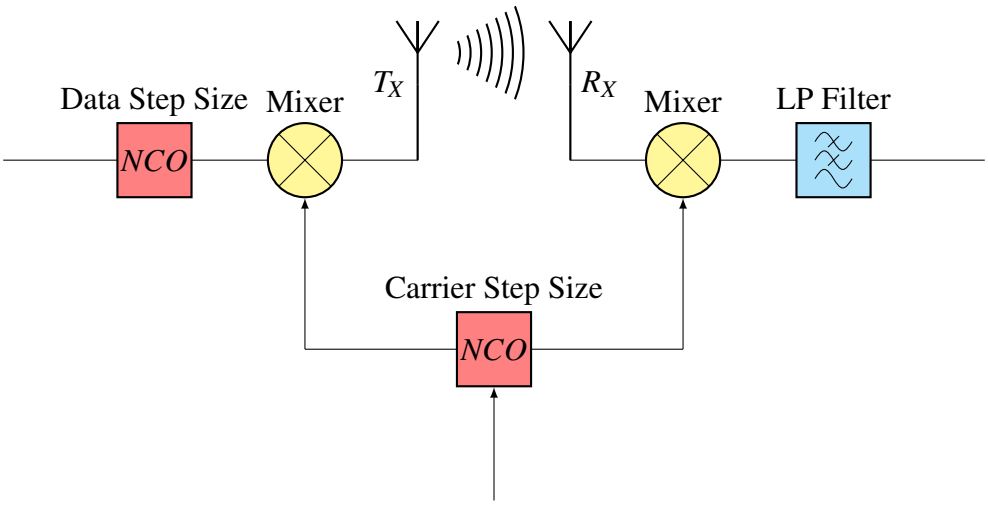

Figure 5: High Level Overview of AM Transmitter and Receiver System

An amplitude modulated signal can be described as the product of a low frequency data signal, $D(t)$, and a carrier signal, $C(t)$. The notation for the data signal is shown in Equation 3

$$D(t) = Acos(2\pi f_d t) \tag{3}$$

With amplitude A, and frequency, f_d. Similarly, the carrier signal is defined as:

$$C(t) = cos(2\pi f_c t) \tag{4}$$

Following on from this, the modulated signal, $S(t)$, can then be derived using relatively simple trigonometry.

$$\begin{aligned} S(t) = D(t) \times C(t) &= Acos(2\pi f_d t)cos(2\pi f_c t) \\ &= \frac{A}{2}\left[cos(2\pi(f_c - f_d)t) + cos(2\pi(f_c + f_d)t)\right] \end{aligned} \tag{5}$$

The need for filtering after demodulation can be explained through some simple mathematics. To demodulate an AM signal, the incoming signal is multiplied again by the original carrier frequency.

$$\begin{aligned} Y(t) &= S(t) \times C(t) \\ &= \frac{A}{2}cos(2\pi f_c t)\left[cos(2\pi(f_c - f_d)t) + cos(2\pi(f_c + f_d)t)\right] \\ &= \frac{A}{2}cos(2\pi f_c t)cos(2\pi(f_c - f_d)t) + \frac{A}{2}cos(2\pi f_c t)cos(2\pi(f_c + f_d)t) \\ &= \frac{A}{4}cos(2\pi(2f_c - f_d)t) + \frac{A}{4}cos(2\pi f_d t) + \frac{A}{4}cos(2\pi(2f_c + f_d)t) + \frac{A}{4}cos(2\pi f_d t) \\ &= \frac{A}{2}cos(2\pi f_d t) + \frac{A}{4}\left[cos(2\pi(2f_c - f_d)t) + cos(2\pi(2f_c + f_d)t)\right] \end{aligned} \tag{6}$$

At the end of the demodulation stage, there are two components left in the signal, the original, baseband frequency component, and the left over modulated component at the carrier frequency. Therefore, a low pass filter can be implemented to remove any of the unwanted carrier signal left after demodulation.

3 Project Methodology

3.1 Design Philosophy

The system design featured a transmitter and receiver setup, linking the DAC and ADC via an SMA coaxial RF cable. This approach is commonly employed as an introductory step in SDR development, as it provides a strong foundation in radio architecture and the associated development cycle. The design was subsequently validated on both the BittWare RFX card and the RFSoC 2x2 development board. Multiple platforms were utilised to leverage the advantages of the 2x2 board's simpler PS interface, PYNQ. PYNQ offers significantly more online documentation, theoretically enabling a faster and more efficient development process compared to PetaLinux. Unlike PetaLinux, where each image is tied to a specific hardware configuration, PYNQ's overlays allow for greater flexibility, facilitating a more adaptable development workflow. Figure 6 shows a high level block diagram of the RFSoC in loopback configuration.

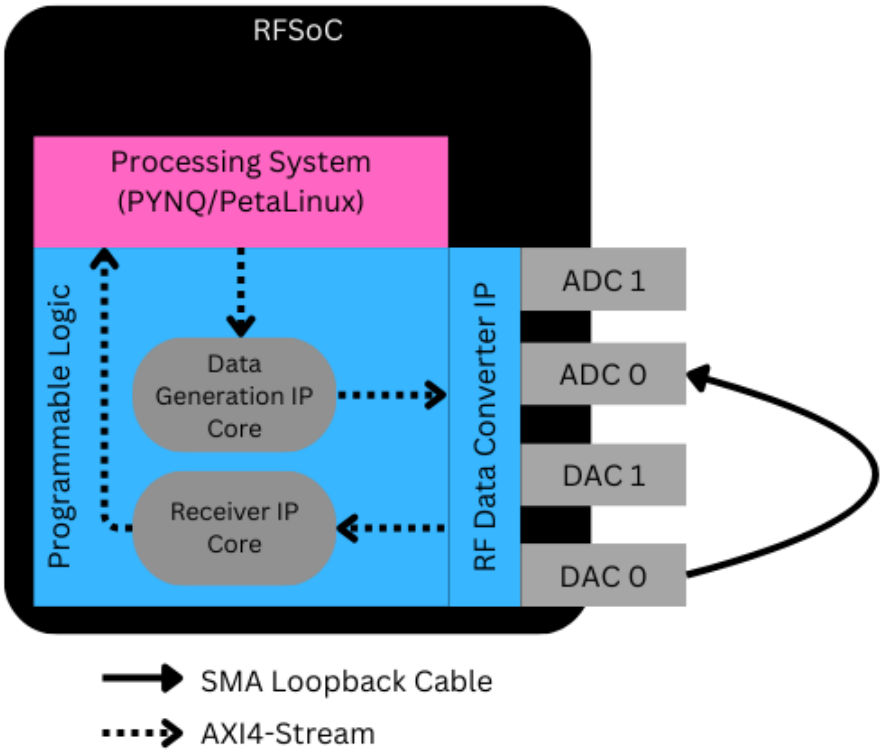

Figure 6: High Level Block Diagram of RFSoC Loopback Configuration

3.2 Digital Logic Design

Primarily, the digital logic was designed using Vitis Model Composer through MATLAB and Simulink. This software abstracts the need to extensively write VHDL code, which can be very tenuous and time consuming, by using a number of pre-defined "blocks", ranging from simple adders, to more complex

operations such as the Fourier Transform. Enabling the AXI4-Stream interface within custom designs can be a challenging task in Model Composer, so to simplify the process to improve development times, an AXI4-Stream template was adapted to fit this design. This template was designed and open-sourced by David Northcote at StrathSDR [15].

3.2.1 Transmitter Design

The transmitter was designed as a "data generation" block, which is then sent to the RFDC. To verify the functionality of the design, the transmitter would generate a sine wave through a Numerically Controlled Oscillator (NCO). The NCO subsystem used in both the transmitter and receiver design is shown in Figure 7.

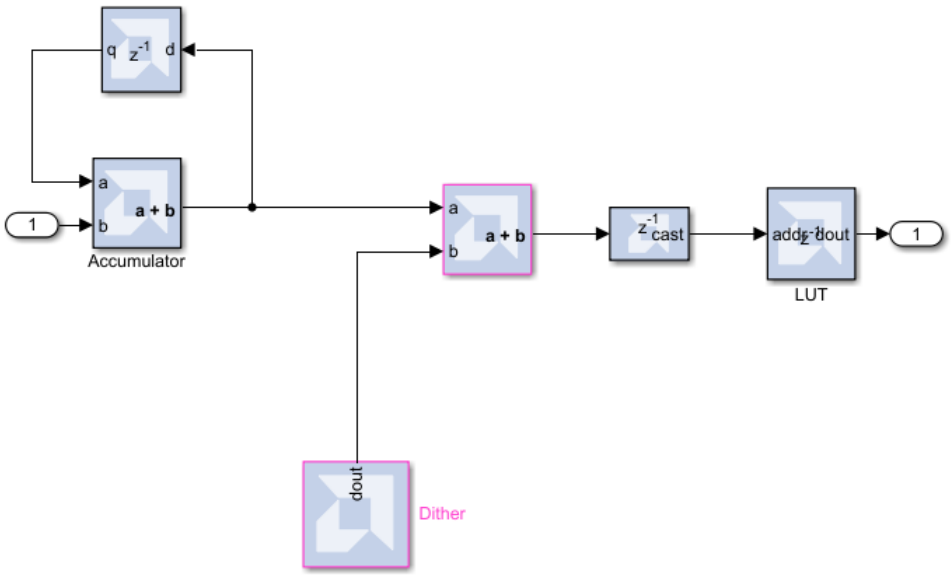

Figure 7: NCO in Vitis Model Composer

NCOs are comprised of two main components, a Look Up Table (right), which is a form of Read Only Memory (ROM), along with an accumulator (LUT) (left). The accumulator is implemented in Model Composer using two components, an adder and a delay. The adder counts up a certain rate, depending on the inputted step size due to the output being fed back to the input. For example, if the input to the accumulator was 1, then the accumulator would output all integer values from 0 up to the wordlength of the output data. The LUT stores, in this case, quantised values of a sinusoid, which are indexed by the output of the accumulator. Along with the two main components of this NCO, there is an extra block which adds "dithering" to the system. This is when a low level of noise is added to the system to improve its Spurious Free Dynamic Range (SFDR) [16], a common figure of merit for RF systems. SFDR is defined as the ratio of the strongest signal present to the largest spurious signal present [17].

As mentioned above, the input to an NCO is called the step size, which determines the set frequency that

the NCO can output. Step size can be calculated using a number of defined system parameters through Equation 7.

$$\mu = N\frac{f_d}{f_s} \tag{7}$$

Where N is the depth of the LUT, f_d is the desired output frequency of the NCO, and f_s is the sampling frequency of the system. The accumulator of the NCO generates n number of address bits which then determines the depth of the LUT through Equation 8.

$$N = 2^n \tag{8}$$

The transmitter design features two NCOs, one which generates the data signal, and one for the carrier signal. Modulation, in this case, refers to carrier modulation, which is when a signal is multiplied by a carrier frequency to be transmitted across a channel. The other type of modulation is called baseband modulation, and refers to the transformation of digital bits to "symbols" in a communications system. The transmitter design in Model Composer can be seen below in Figure 8.

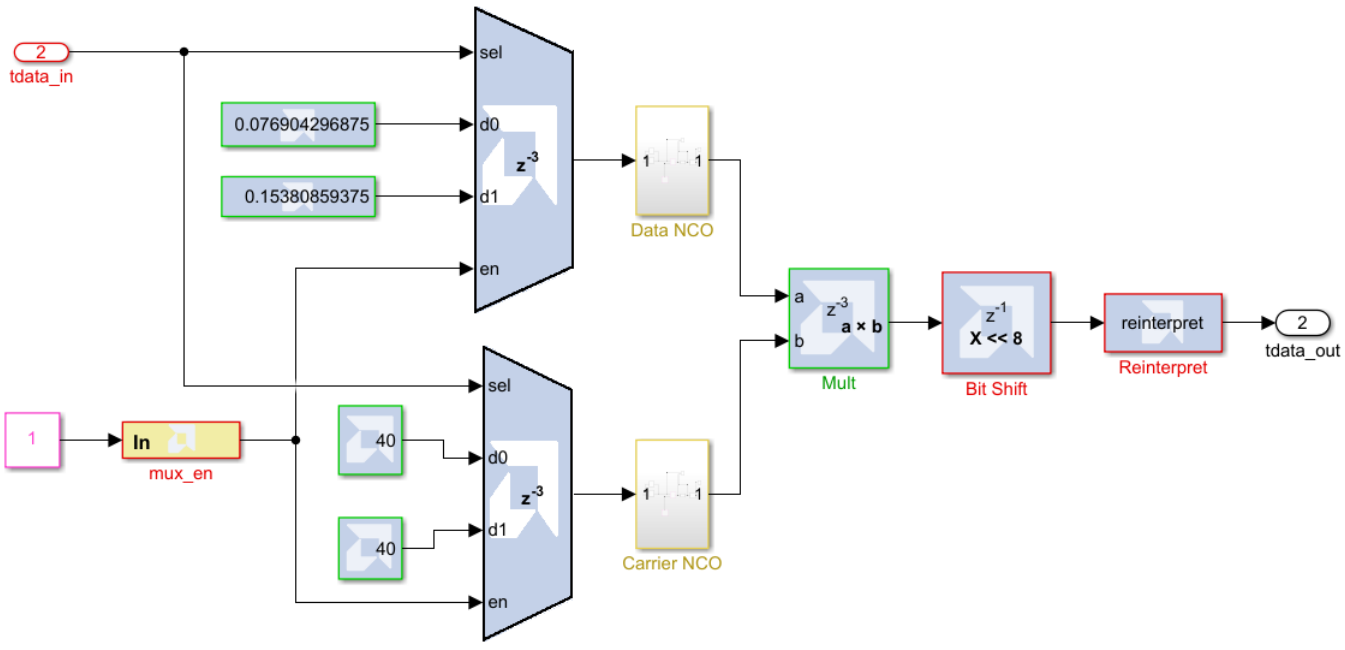

Figure 8: Transmitter Design in Vitis Model Composer

The first input to this design (*tdata_in*), seen at the top of Figure 8, is streamed from the PS and selects the NCO frequency to output through a multiplexer (MUX). The second input is used to essentially turn the NCOs on and off. This is done through an enable pin that only allows the MUX to pass the step size input when the signal *mux_en* is set high (1). For simplicity at the receiver side, only one carrier frequency was used for both values of tdata_in. The enable pin was configured as an AXI4-Lite register, therefore,

could be turned on and off through simple read and write operations using PYNQ or PetaLinux. Each NCO outputs 8 bit data with the binary point set at 4, meaning 4 integer bits and four fractional bits. The output of each NCO is sent into the multiplication (mixer) block, which is set to output at **full** precision, rather than user-defined. The full precision of the mixer stage outputs 16 bit data with the binary point set at 8, hence why the bit shift is set to move the binary data 8 bits to the left. The **reinterpret** block is used to change the signed data coming from both NCOs into unsigned data, which is needed in order to interface properly with the AXI4-Stream template.

The MUX block has a minimum of two inputs, hence the reason for having two transmission frequencies, this can of course be changed as needed if more frequencies are required. The data frequencies were chosen to be 100 kHz and 200 kHz. Equation 7 was used to calculate the step sizes of each NCO. Note that the depth of the LUT inside of the NCO was chosen to be a standard value of 256 bits, whilst the sampling frequency was chosen to be 333 MHz (this was chosen for a specific reason which will be discussed later in the report).

$$\begin{aligned}\mu_1 &= N\frac{f_d}{f_s} \\ &= 256 \cdot \frac{200 \cdot 10^3}{333 \cdot 10^6} \\ &= 0.07688\end{aligned} \qquad \begin{aligned}\mu_c &= N\frac{f_c}{f_s} \\ &= 256 \cdot \frac{52 \cdot 10^6}{333 \cdot 10^6} \\ &= 40\end{aligned}$$

Since the second frequency is double that of the first, the step size needed to output 200 kHz simply needs to be doubled ($\mu_2 = 0.15376$). The carrier frequency was chosen to be approximately 50 MHz, however, this was originally much higher but was chosen to be decreased to fit the lower sampling frequency needed to interface with the BittWare reference design. Since a step size of 40 gives approximately 50 MHz, this was ultimately chosen to be the preferred carrier frequency.

3.2.2 Receiver Design

The Receiver is designed to essentially reverse the operations that the transmitter does to the signal. This means that the incoming signal is demodulated, and low-pass filtered. The model for the receiver in Model Composer is shown in Figure 9.

The receiver takes the input of the RF-ADC and demodulates it with an NCO set to output the same carrier frequency as the transmitter. Following on from the initial demodulation stage is the filtering stage, which is comprised of a low-pass Finite Impulse Response (FIR) filter, which was implemented into the FIR compiler block from Model Composer. The filter coefficients were designed using MATLABs **fir2()** function, which takes a number of arguments to specify the order, cut off frequency, and magnitude characteristics. The magnitude and phase response of this filter can be seen below in Figure 10.

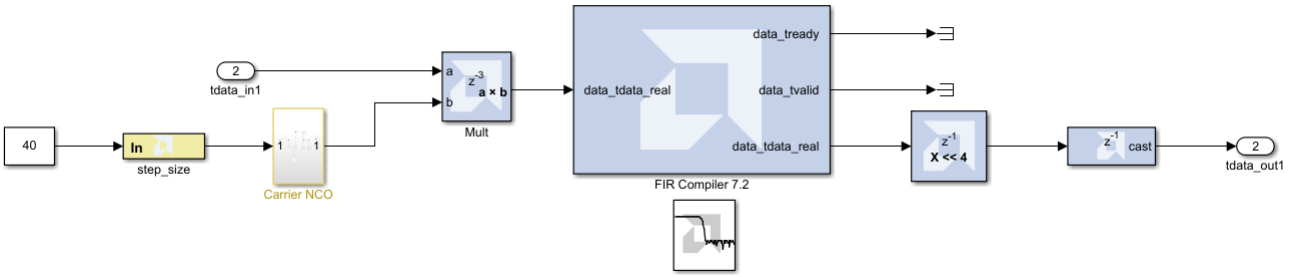

Figure 9: Receiver Design in Vitis Model Composer

Similarly to the transmitter, the receiver NCO generates 8-bit data with a binary point at 4. The multiplication block is once again configured to output full precision. Given that the incoming signal is 16 bits, the mixer produces a 20-bit output with a binary point at 4. The FIR Compiler block outputs a wider data width than its input for several reasons. Primarily, the FIR filter operates using multiply-accumulate operations, where the input signal is multiplied by a series of weights. With each accumulation, the bit width increases to accommodate potential overflow, potentially preventing any inaccuracies. The FIR Compiler block outputs 39 bit data with a binary point at 4, so there is a shift block before the output to set the binary point to 0. Therefore, it was decided that the receiver would output 32 bit wide AXI4-Stream data, to prevent a significant loss of data accuracy.

3.2.3 IP Integrator Design

Model Composer is a great tool for testing the functionality of a hardware design through simulations, but to actually implement the designs, HDL code must be generated using the system generator tool. This tool allows a user to specify the target hardware device (or chip) and generate custom code, either in VHDL or Verilog. This can then be exported as IP, and used in Vivado's IP Integrator (IPI) tool. Since Vivado projects are specific to the target hardware, separate projects for the RFSoC 2x2 and the RFX were needed, due to the difference in both chip generations and design goals.

3.2.3.1 RFSoC 2x2 Design Validation

Given the lack of accessibility of the RFX, it was decided that the best approach for testing and developing this project was to use the RFSoC 2x2 development board as a means to work on the project whilst unable to visit the BittWare office. The RFSoC 2x2 is shown in Figure 11.

The 2x2 was used to verify the functionality of the transmitter and receiver designs from Model Composer on real hardware through a loopback configuration. Custom IP generated from Model Composer can be added by pointing to the "ip" folder (found inside the "netlist" folder of the Model Composer project) in Vivado. After IP is added, it can be used in any block design and connected to a number of proprietary IP blocks that come standard with Vivado. Figure 12 shows the Loopback configuration model on the RFSoC

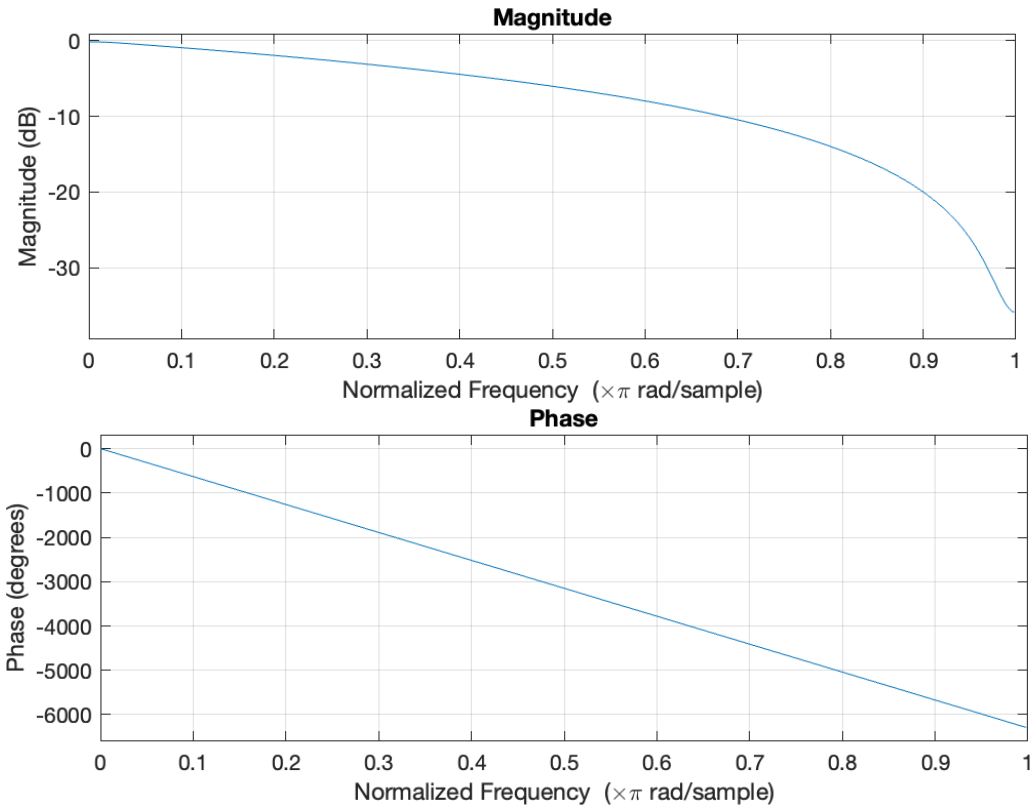

Figure 10: Magnitude and Phase Response of Low Pass FIR Filter

2x2 in Vivado's IP integrator.

To read from the output of the receiver, and write to the input of the transmitter, an AXI Direct Memory Access (DMA) controller was used. The AXI DMA is instructed by a processor core, in this case the PS, to move data between points in the design [6]. The DMA is used for accessing the shared memory between the PS and PL, allowing for high speed data transfers between different IP. At the core of this design is the ZYNQ Ultrascale+ RF Data Converter, which is the equivalent IP core of the RFDCs previously mentioned. The RFDC IP core combines the RF-ADC and RF-DAC in one single package, with data being inputted to the DAC side, and received from the ADC side.

Configuring the RFDCs in Vivado takes significant design effort, as the master and slave sides can potentially require different clocking speeds, which complicates design efforts as the correct clock has to be meticulously connected to each component on both the transmit and receive side. It is best practice to clock everything on the transmit side with the same frequency as the RF-DAC, and do the same for the receive side with the RF-ADC. Doing so limits the risk of crossing a clock domain, which can make the design fail timing requirements (an important metric in FPGA programming) and have a plethora of issues in practice, such as DMA crashes and improper AXI transactions. A clock domain is a region of the FPGA that operates from a single clock source [19]. In this design, there are three clock domains, one for the transmit side, one for the receive side, and another for the PL. The RFDCs have a built in mixer, which can be utilised to implement further modulation up to extremely high frequencies. However, this mixer was not

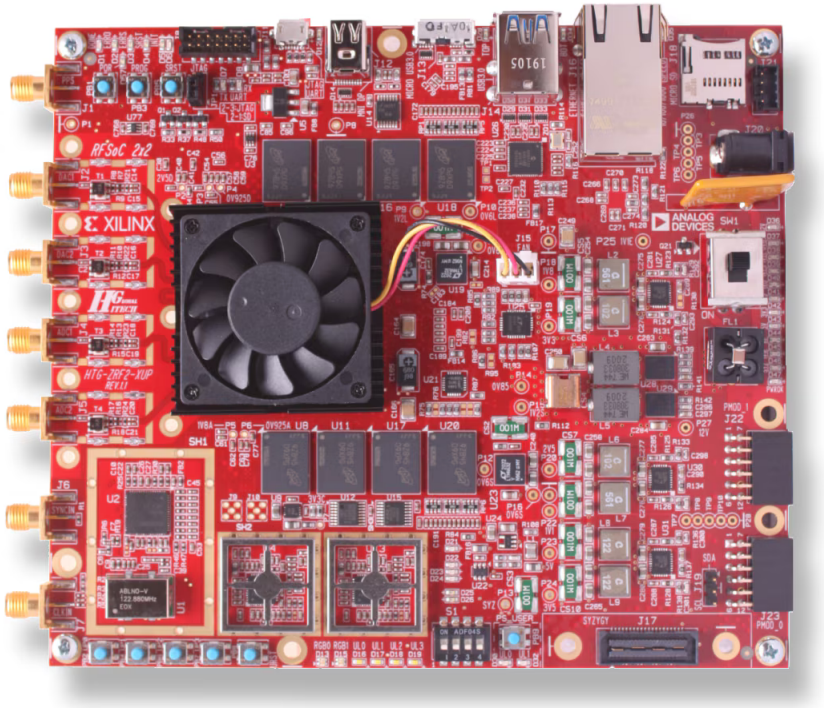

Figure 11: Xilinx RFSoC 2x2 Development Board [18]

used, this was to demonstrate the signal generated in hardware, along with the custom mixer that modulates and demodulates the signal. After the RFDCs were configured, the ADC and DAC output clocks were then connected to their respective parts of the design for transmitting and receiving. The RF-ADC could only output a 64 MHz output clock, which is significantly slower than the required 128 MHz. To solve this issue, a **Clocking Wizard** IP was used to increase the clock frequency to its required value. Table 2 summarises the RFDC clock settings for the RFSoC 2x2 design.

Tile	Sampling Rate (GSPS)	Max Fs (GSPS)	PLL	Reference Clock (MHz)	PLL Ref Clock (MHz)	Ref Clock Divider	Fabric Clock (MHz)	Clock Out (MHz)
ADC 224	1.024	4.096	✓	204.800	204.8	1	128.000	64.000
ADC 225	2.0	4.096	-	2000.000	-	1	0.0	15.625
ADC 226	2.0	4.096	-	2000.000	-	1	0.0	15.625
ADC 227	2.0	4.096	-	2000.000	-	1	0.0	15.625
DAC 228	1.024	6.554	✓	204.800	204.8	1	128.000	128.000
DAC 229	6.4	6.554	-	6400.000	-	1	0.0	50.000

Table 2: RF Data Converter Clock Settings

Notice that there are two “sub-blocks” within this design, with one for the transmit side, and another for the receive side. This is used to mitigate the crossing of clock domains inside of the hardware design. Figure 13 shows the transmit side sub-block.

It must be noted that there is now two DMA IP cores within this design, one specifically for reading, and another for writing. In practice, this would operate by sending a DMA buffer (a certain amount of allocated memory for the DMA IP to access through read and write channels on the PS) to the custom transmit IP,

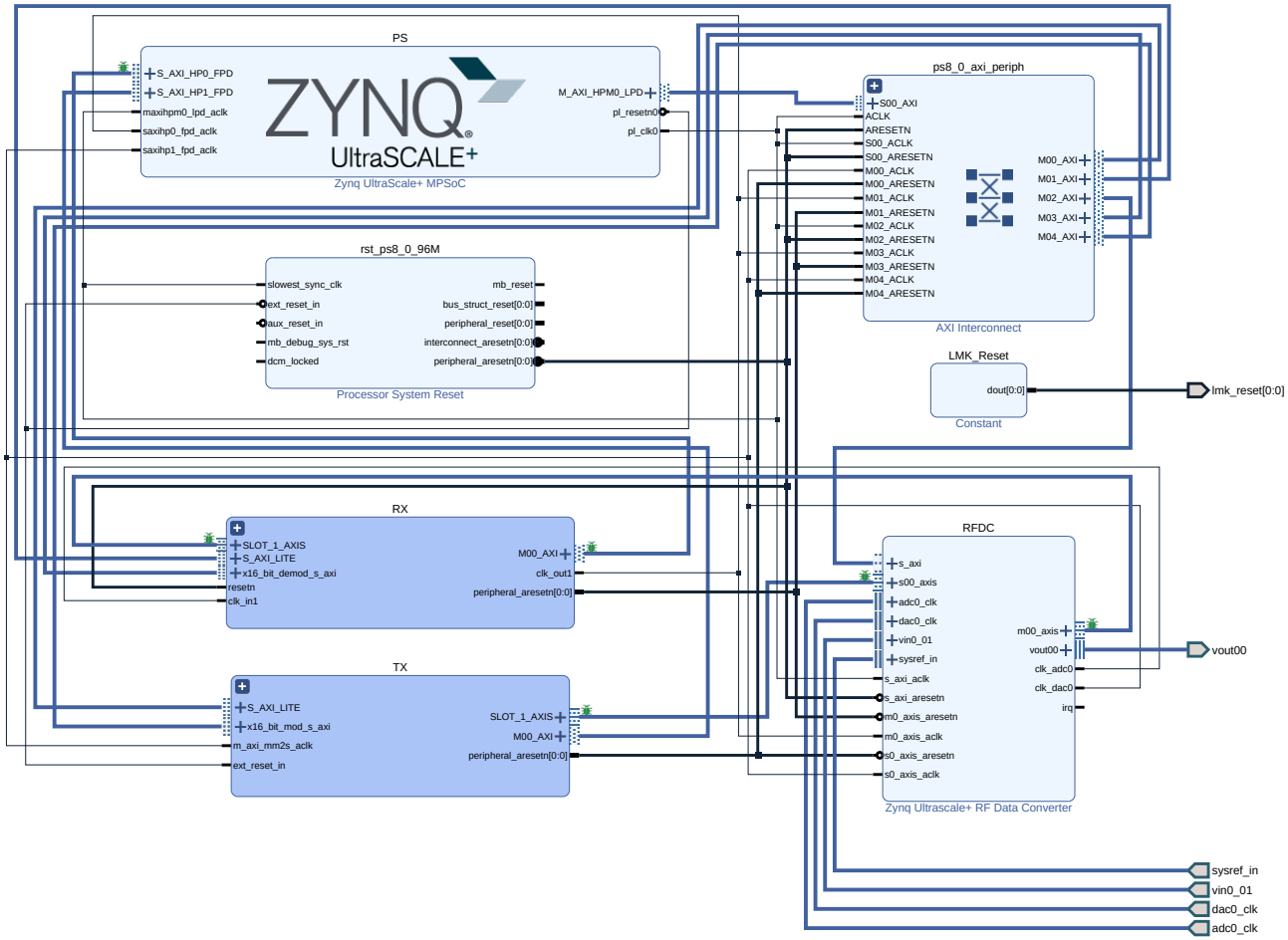

Figure 12: Vivado IP Integrator Design for RFSoC 2x2 Loopback Configuration

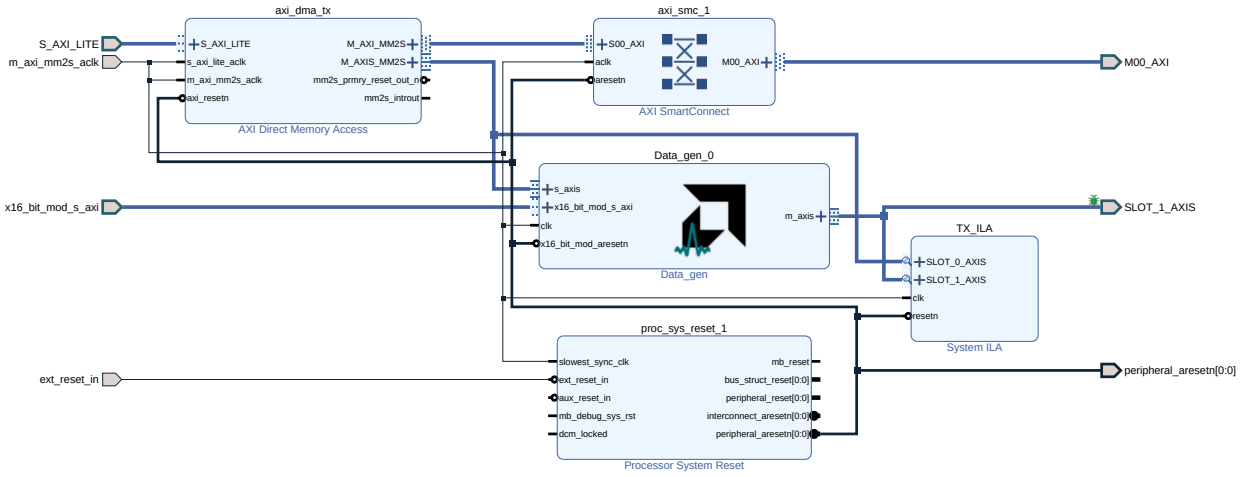

Figure 13: Transmit Side Sub-Block Inside Vivado IP Integrator

which then starts the NCO to send a signal. In the software, the DMA buffer would then be read from the output of the receive side DMA, confirming the loopback configuration. The two sub-blocks were created to completely separate both the ADC and DAC clocks, therefore reducing any problems that can occur

when crossing clock domains.

The RFSoC 2x2 is no longer manufactured and has since been replaced by the newer RFSoC 4x2, this means that using newer versions of Vivado can be difficult since the board files are not included within the Vivado package. The board files can be independently used in Vivado through some Tcl commands that can be found on the RFSoC 2x2 website [20]. Due to the discontinuation of the 2x2, there is a slightly limited amount of support and documentation online when compared to the newer 4x2 or other development boards.

3.2.3.2 BittWare RFX-8440 2x100GbE Design Implementation

The Design implementation for the RFX is slightly different to that of the RFSoC 2x2, as the custom transmitter and receiver were instantiated as components inside of a significantly larger design. When shipping their cards, BittWare includes a number of "reference" designs, which are used to help customers understand and test the operation of the card, without having to design systems completely from the ground up. This design approach was taken for this project, as the previously mentioned transmitter and receiver were implemented within a larger design, rather than a fully custom one. The reference design in this case is the RFX8440-2x100GbE, which configures the entire FPGA, including all RFDC channels, the gigabit transceivers, and the PS. The project is aptly named because it uses the two 100 gigabit Ethernet Quad Small Form-factor Pluggable (QSFP) transceivers, which can be connected to the card for high speed optical communications. The block diagram from BittWare's framework logic guide of the 2x100GbE design can be seen in Figure 14 [21].

This design is highly complex, since it features all components of the FPGA in a single bitstream. Inside of the design, and in Figure 14, is the DSP placeholder block, which has been designed for customers or other engineers to implement there own form of DSP without having to reconfigure the entire design with all of the I/O. Despite what Figure 14 shows, the DSP placeholder is not an actual block inside of the Vivado IP integrator design that can be replaced with the users IP, but rather a VHDL file named `ii_dsp_top.vhd`. Functionally, this file connects the RFDCs together, involving no DSP at either the DAC or ADC side. The reference designs are inside a folder that can be downloaded or purchased by customers that may be interested in starting their designs from a known point. Building the Vivado project involves navigating to the 2x100GbE project folder inside of the Vivado Tool command language (Tcl) console [22], then running the `Build_Vivado.tcl` script. This script launches the project and builds the block design(s) with the necessary IP added, and connections already made, a process which can take up to an hour.

Implementing custom IP inside of the 2x100GbE design is slightly different to that of the 2x2 design, as it is not added inside of a block design, but rather through a component instantiation using VHDL. When the transmitter IP is added to the project, its output products are generated, which includes an instantiation template inside of the `.vho` file. VHDL components are how separate top level entity declarations can be re-used inside of other designs. The component definition for the transmitter design can be seen in Listing

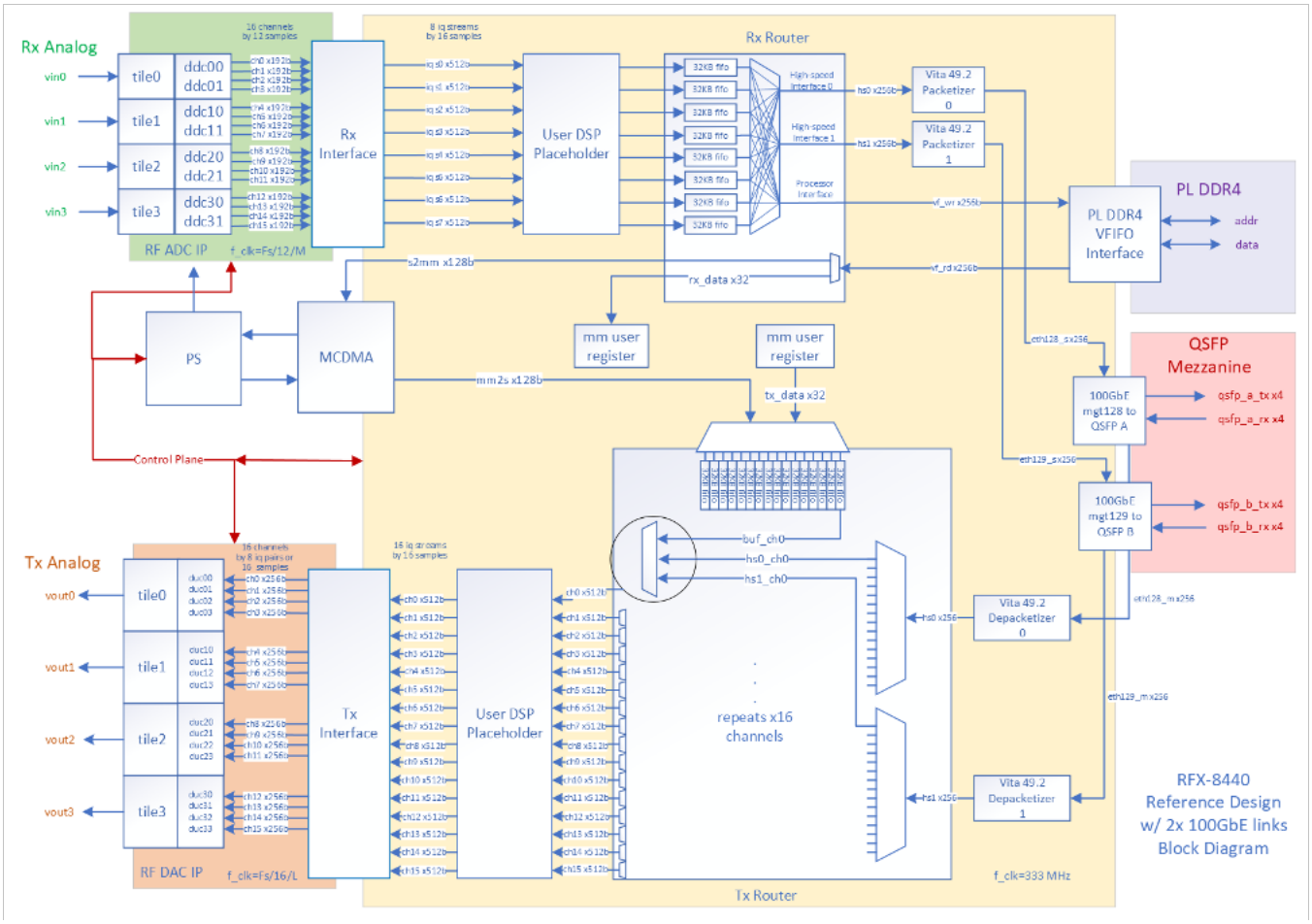

Figure 14: Block Diagram of BittWare RFX-8440 With 2x100GbE Links

1.

```
COMPONENT DUT_data_in_16bit
PORT (
    s_axis_tvalid : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    s_axis_tdata  : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    s_axis_tlast  : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    m_axis_tready : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    mux_en        : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
    clk           : IN STD_LOGIC;
    m_axis_tvalid : OUT STD_LOGIC_VECTOR(0 DOWNTO 0);
    m_axis_tdata  : OUT STD_LOGIC_VECTOR(15 DOWNTO 0);
    m_axis_tlast  : OUT STD_LOGIC_VECTOR(0 DOWNTO 0);
    s_axis_tready : OUT STD_LOGIC_VECTOR(0 DOWNTO 0)
);
END COMPONENT;
```

Listing 1: Transmitter Component Definition

After defining the component inside of the **dsp_top** entity, it can then be instantiated, as in Listing 2.

```
transmit_component : DUT_data_in_16bit
PORT MAP (
    s_axis_tvalid => "1",
    s_axis_tdata => nco_driver,
    s_axis_tlast => "1",
    m_axis_tready => m_axis_tx_tready(0 downto 0),
    mux_en => nco_mux_en,
    clk => axis_clk,
    m_axis_tvalid => m_axis_tx_tvalid(0 downto 0),
    m_axis_tdata => m_axis_tdata_temp,
    m_axis_tlast => m_axis_tx_tlast(0 downto 0),
    s_axis_tready => open
);
```

Listing 2: VHDL Transmitter Component Instantiation

The component instantiation is what actually includes the component in the design, where the signals from the 2x100GbE design are connected to the inputs and outputs of the custom transmitter. The 2x100GbE design configures all of the RF-ADC and RF-DAC channels, however, the goal is to only transmit over one channel, which means the transmitter component needs to take over one of these channels. The RFDC channel on the RFX uses a 512 bit array, which has been custom created for this reference design. Since the custom transmitter only outputs 16 bit data, the signal had to be manipulated in order for the two to match. To achieve this, the output of the transmitter IP was concatenated with itself 32 times to achieve the correct buffer width of 512 bits. This concatenation was then assigned to the input of the RFDC channel, `m_axis_tx_tdata`.

Following the definition and integration of the transmit component, there must be a number of registers added to control the new IP. This is done through another file inside of the reference design, `ii_dsp_regs.vhd`, which configures all of the necessary registers for the dsp_top entity within the memory map, but also has space to add custom registers for any user applications. For this design, only two registers were added, one for controlling the NCO output frequency, and another for the enable pin connected to both NCOs. To do so, the name of the register must first be created inside of the dsp_regs entity declaration as an input of 1 bit, the register is then defined at the bottom of the file, using the other registers as a template. Listing 3 shows this register definition.

The variable `reg_init` defines the initial value that the register is mapped to. For the NCO register (`nco_driver`), this can be arbitrarily set to either zero or one, as long as the enable pin (`mux_en`) is initialised to zero, which it was.

TALK ABOUT DATA PACKETISATION - AND ONLY GETTING TRANSMIT COMPONENT WORKING

```
nco_driver                  <= reg_o(2)(0 downto 0);
reg_init(2)(0)              <= '1';              -- NCO controller
reg_i(2)                    <= reg_o(2);

mux_en                      <= reg_o(3)(0 downto 0);
reg_init(3)(0)              <= '0';              -- Enable pin for mux 1 and 2
reg_i(3)                    <= reg_o(3);
```

Listing 3: VHDL Register Definition

3.3 Processing System Design

Although the design of the PL is crucial to the physical operation of the FPGA, the PL design is equally important for interfacing with and monitoring SoC devices through various development kits, such as PYNQ and PetaLinux.

3.3.1 PYNQ

Beginning with the PYNQ design for the RFSoC 2x2, this is created using a Jupyter notebook that interfaces with various Python libraries. After building the project for the 2x2 and generating the bitstream, the block design was exported to the default file locations. To obtain the overlay efficiently, a shell script was written to avoid manually navigating the file system to locate the `.bit` and `.hwh` files. The script also saves the overlays into the **Overlays** folder while retaining a copy of the files in their original location. The two files must have the same name for PYNQ to recognise the overlay; the script ensures this requirement is met. After loading the overlay onto the board, the IP contents of the bitstream can be inspected using the two lines of code found in Listing 4.

```
ol = Overlay("Design_1_wrapper.bit")
ol?
```

Listing 4: Python Code for Inspecting Overlay Content

The question mark operator exports the contents of the bitstream into the jupyter notebook environment to ensure the right overlay is being used. From there, the controllable IP cores can be assigned to a Python variable, which allows for more readable code when calling different functions inside of the notebook.

After loading the overlay onto the FPGA, it is necessary to configure the reference clocks in software before initiating any PS–PL communication. The two required reference clocks for configuring the 2x2 board are the LMX and LMK clocks. These reference clocks are aptly named because they provide a high precision, low jitter clock for the RFDCs to use as a reference when generating their own clocks. Jitter refers to the

variation in clock period between two consecutive cycles over a span of 1000 cycles [23]. The required frequencies for the LMK and LMX clocks can be found in the Xilinx PYNQ GitHub repository, specifically in the **xrfclk** package, which also includes a built in function to set the reference clocks with user defined frequencies, according to the board.

Although the RFDCs are pre-configured in hardware, they must also be properly initialised in software for the design to function correctly. Extensive resources are available online for configuring RFDCs, with the StrathSDR GitHub page being a particularly valuable source, offering numerous open-source projects that utilise RFSoC and configure it through PYNQ. This is achieved using the **xrfdc** Python package, which provides an interface to the RFDCs, ensuring proper clock configuration, dynamic Phase-Locked Loop (PLL) adjustments without requiring a bitstream reload, and setting mixer parameters along with Nyquist zones. The data converter settings are configured in two Python function declarations, but can be altered by the user to specify different sampling frequencies, or ADC/DAC tiles. The settings for the RFDCs have the mixer configured as a fine mixer that outputs 0 Hz to allow for correct DAC and ADC mixing. Even though no mixer was used in the hardware configuration, the older generation RFSoCs expect some mixer settings to be applied in software to allow for correct configuration so this was chosen since it would not interfere with the incoming signal and still allow for data transmission.

Following on from this, the PS-PL communication could be established. This was done through the assignment of the both the DMAs and RFDC to a Python variable, which could be called back later when transferring data. The exact name of the IP that is assigned to the variable can be found through the previously mentioned (?) operator under the **IP Blocks** section. The variable is then assigned through the use of the (.) operator to assign a new variable to specific IP. Listing 5 shows the assignment of a variable *tx_dma* to the transmit side DMA block.

```
tx_dma = ol.TX.axi_dma_tx
```

Listing 5: Python DMA Variable Assignment

Finally, the built-in **Allocate** package is used to allocate DMA buffers in memory which can then be transferred between PS and PL. In this design, both the send buffer and the receive buffer have a size of 4000 samples (the maximum at which PYNQ allows), to ensure clarity when receiving data. The data can then be sent using the `transfer()` function on both the send and receive DMA channels. These lines of code are then followed by the `wait()` function which waits until the DMA channel goes back to an idle state. Following on from this, the received DMA buffer can be plotted using the **matplotlib** package, a comprehensive tool for data visualisation and plotting.

3.3.2 PetaLinux

PetaLinux has a more complex deployment process, as a PetaLinux image needs to be accompanied by a specific bitstream. This means that testing and confirming design choices can take significantly longer when working with a board that does not support PYNQ. BittWare thankfully has a set build process for creating and customising PetaLinux images through some documentation and build scripts. However, the RFX needs specific hardware settings to configure properly, specifically on the PS, which needs to have a number of different ports enabled for communication with the images boot file system. This means that creating completely custom images in PetaLinux can be a long and painstaking process, which takes significant design effort and time. This is another reason as to why the reference design was used, as the hardware design was properly configured to allow the PetaLinux image to build and run properly, as it had been through significant testing throughout its design stage.

The first step in creating the PetaLinux image is to export the hardware image (bitstream) to a `.xsa` file, saved in the specified PetaLinux folder within the larger reference design folder. This xsa file is slightly different to the overlays seen in PYNQ development, as it features a number of different files inside which tell the PetaLinux build about the hardware design - rather than the two overlay files that PYNQ needs to configure hardware. Although, newer PYNQ releases do feature support for xsa files, which streamlines the development process slightly, since the `.bit` and `.hwh` files do not need to be found within the file structure of the Vivado project.

After exporting the hardware to the appropriate file, the next step is to source the PetaLinux build tools. This was accomplished by locally downloading the latest version of PetaLinux (2024.1) at the BittWare office, which stored all necessary build files on a local laptop. The Linux `source` command was then used to execute the `settings64.sh` file, which sets up the PetaLinux build environment by configuring various environment variables and folder paths. Once the tools are sourced, the `build_petalinux.sh` script is run to initiate the entire build process using a series of PetaLinux commands (e.g., `petalinux-build`) that configure the image.

3.3.2.1 Loading PetaLinux Images onto the RFX

Once the build process has completed, the PetaLinux image has to be loaded onto the card. There are a number of files that are needed to load the image onto the card, these files are summarised in Table 3.

File	Description
BOOT.bin	Binary file containing bitstream contents and FPGA configuration
Boot.scr	U-Boot boot script
image.ub	Open-sourced Linux boot loader used for ZYNQ 7000 processors [24]
zynqmp_fsbl.elf	First stage boot loader

Table 3: PetaLinux Image Files

BittWare offers an optional utilities package for the RFX, called **rfxutils**, which provides example Python scripts for running on the RFX, along with boot scripts for modifying PetaLinux images on the card. The files listed in Table 3 were copied into the `flash_images/` folder, which contains these boot scripts. The PetaLinux image was then flashed onto the card using a Joint Test Action Group (JTAG) flash. JTAG is a standardised method for testing interconnects on Printed Circuit Boards (PCBs) after assembly; however, it is also commonly used for programming FPGAs and flash devices [25], as is the case here. The JTAG device is connected through a micro-USB adapter which is connected to the motherboard of the host computer. To boot the new image over JTAG, the card must first be set to JTAG mode. This is achieved by setting both switches on the bottom of the RFX to HIGH while the RFX is powered off and disconnected from the PCIe slot of the motherboard. Once the board is in JTAG mode, the Vitis tools need to be sourced in the same manner as the PetaLinux tools. After this step, the `jtag_flash.sh` script can be executed to initiate the boot loading process.

Upon completion of the flash process, the card needs to be put into Quad Serial Peripheral Interface (QSPI) mode. QSPI is a peripheral which allows the interfacing between different flash devices [26], and is the main flash memory of the RFX. The RFX can be put into QSPI mode by setting the left hand side switch to LOW, after which the card can be rebooted. Once in QSPI mode, the card can be accessed via the **minicom** terminal, which communicates with the RFX via a serial connection. One way to verify the image has been successfully loaded onto the card is to inspect the user and hostname at the left hand side of the minicom terminal. If the image has loaded successfully, this should say `root@rfx8440_2x100GbE`, which indicates that the minicom terminal is logged in as the **root** user, and the correct PetaLinux image has correctly been flashed.

3.4 GNU Radio

Throughout the project, GNU Radio played a key role, particularly in the initial stages of development and learning. However, as the project progressed, the focus shifted, and GNU Radio became more of a learning tool rather than a deployment platform. This shift was primarily due to the complexity involved in integrating GNU Radio with novel hardware devices. Nevertheless, GNU Radio remained a valuable resource, offering extensive tutorials on its website. For instance, the Quadrature Phase Shift Keying (QPSK) transceiver tutorial provided a comprehensive understanding of various modulation schemes, demonstrating how GNU Radio can be used to design and implement SDR systems across different hardware platforms.

Expand on this?

4 Results and Analysis

This section presents the results from both software simulations and the various hardware implementations across both RFSoC platforms. It also includes a comparison of the different implementations and an analysis of the discrepancies between simulated and practical results.

4.1 Vitis Model Composer Simulations

When using Model Composer for hardware design on FPGAs, it is always best practice to ensure a design functions as expected in simulation before implementing onto hardware. This ensures that if any issues arise that they can be more easily identified - as debugging at the hardware stage can prove extremely difficult at times.

4.1.1 Transmitter Design

The goal of the transmit component was to generate data in the form of an AM sine wave, and send this through the RFDCs to then be recovered and viewed in the PS. To ensure both of the NCOs are behaving as expected, they can be inspected in both the time and frequency domains. For clarity, the time domain data has exported to the MATLAB workspace, however, the frequency domain data will be viewed through the Model Composer spectrum analyser tool, as this provides a better interface for viewing frequency domain data. To confirm the operation of both NCOs, their time and frequency plots can be seen in Figure 15.

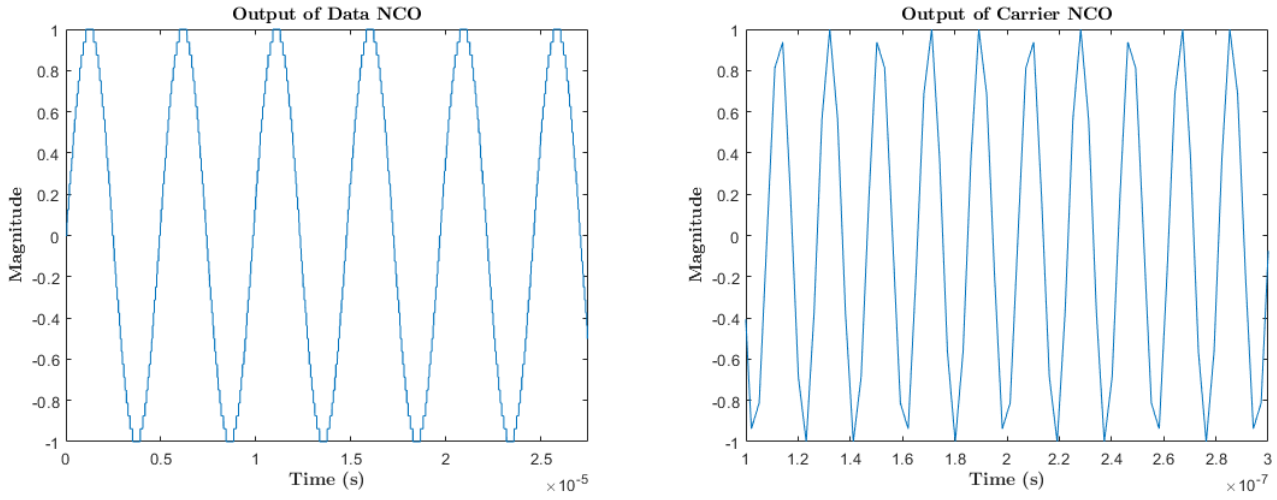

Figure 15: Time Domain Representations of Data NCO and Carrier NCO

Thankfully, both NCOs output a clean sinusoidal waveform, and by inspection of the x-axis it can be seen that the carrier NCO is set to a much higher frequency than the data NCO. The carrier NCO has a less noticeable sinusoidal shape due to the sampling rate of the system, which is only able to represent the higher

frequency with around 6 samples per waveform, whereas the data NCO is represented with a significantly higher number of samples. The best way to investigate which frequency is present in either waveform is through the **Spectrum Analyser** tool, from Simulink. Figure 16 shows the FFT the Data NCO.

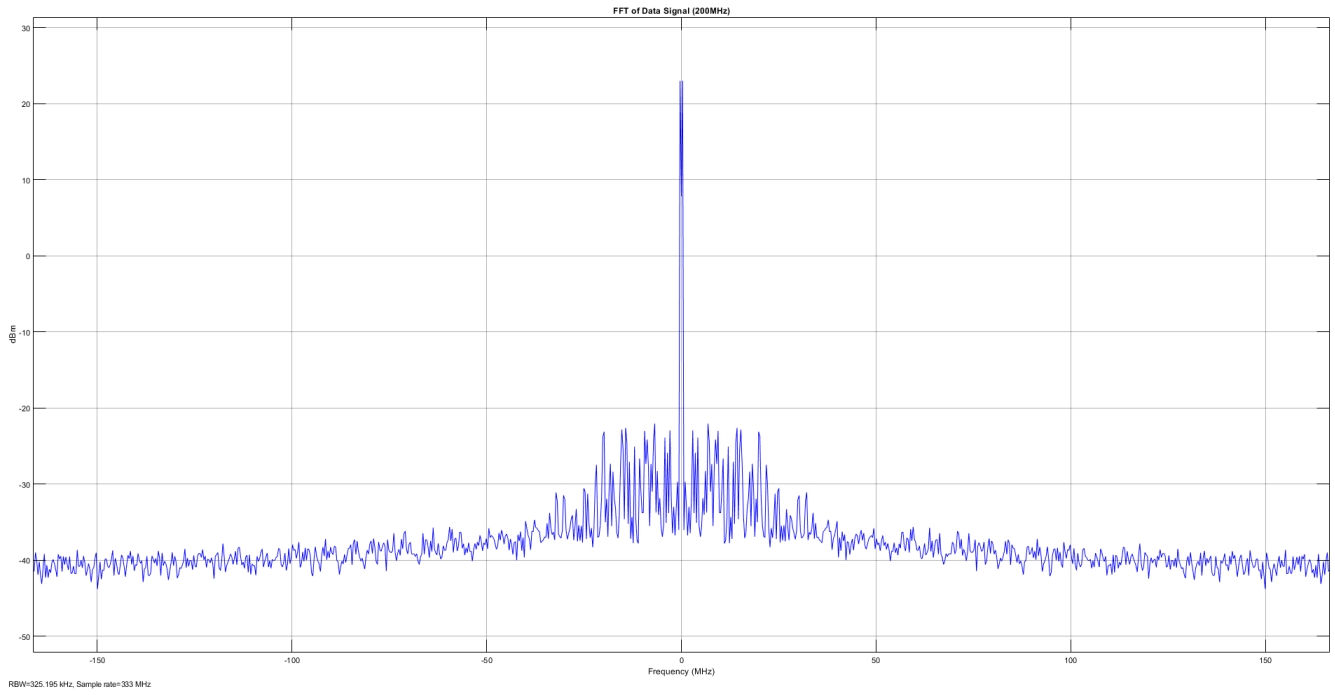

Figure 16: FFT of Data NCO Output Signal

There is a peak at approximately zero, which is unexpected since the output signal should be around 200 kHz, however, further inspection of this graph using the built-in **peak finder** tool shows that there is actually two peaks, just either side of zero. The x-axis is in MHz, so the lower 200 kHz signal is harder to delineate within this spectrum. The two peaks either side of zero appear at ± 0.325 MHz. Again, this is unexpected since investigating the time domain waveform, the frequency can be estimated to accurately represent the desired 200 kHz signal, however, this could potentially be down to the FFT resolution that is present within the spectrum analyser. The FFT resolution defines the width of each frequency bin and is shown in Equation 9.

$$\Delta f = \frac{f_s}{N} \tag{9}$$

Where N is the number of samples present in the signal. The Simulink simulation time can be selected at the top of the screen, and this was set to $10000/fs$, which gives a total of 10000 samples obtained during the simulation. Therefore, the FFT resolution of the spectrum analyser would be:

$$\Delta f = \frac{333 \cdot 10^6}{10000}$$
$$= 33.3\text{kHz}$$

This means that signals that are less than Δf apart cannot be distinguished from each other. Given this, it can be said that the output of the data NCO falls within a larger frequency bin and cannot be directly distinguished in the frequency domain. The frequency domain plot of the carrier NCO can be seen in Figure 17.

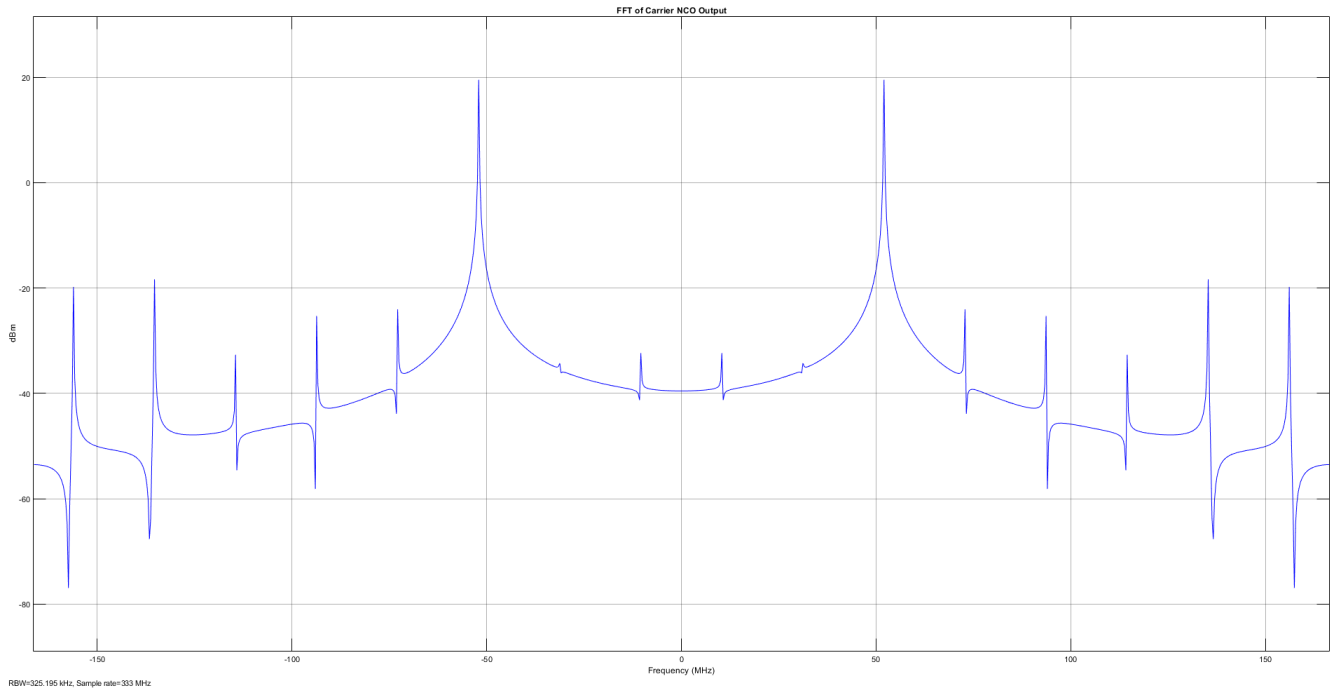

Figure 17: FFT of Carrier NCO Output Signal

Thankfully, the FFT of the carrier signal is much clearer, and both the positive and negative frequency components of this signal can be clearly verified to match the 52 MHz expected output signal. The peak finder tool also verifies this. After confirming the two signals are correctly represented in both the time and frequency domain, they can then be modulated and transmitted. Figure 18 shows the output of the transmitter before the **cast block**.

However, before the signal can be transmitted, it must be converted to an unsigned and fixed-point output, in order to interface properly with the AXI4-Stream template. The output of the cast block is shown in Figure 19.

Of course, this signal is not exactly the same as the desired AM signal that would be preferable to transmit, however this signal is still recognisable and can be identified after the receiver stage. However, zooming in on this signal gives a more detailed view of the transmitted waveform. The frequency domain plot of this

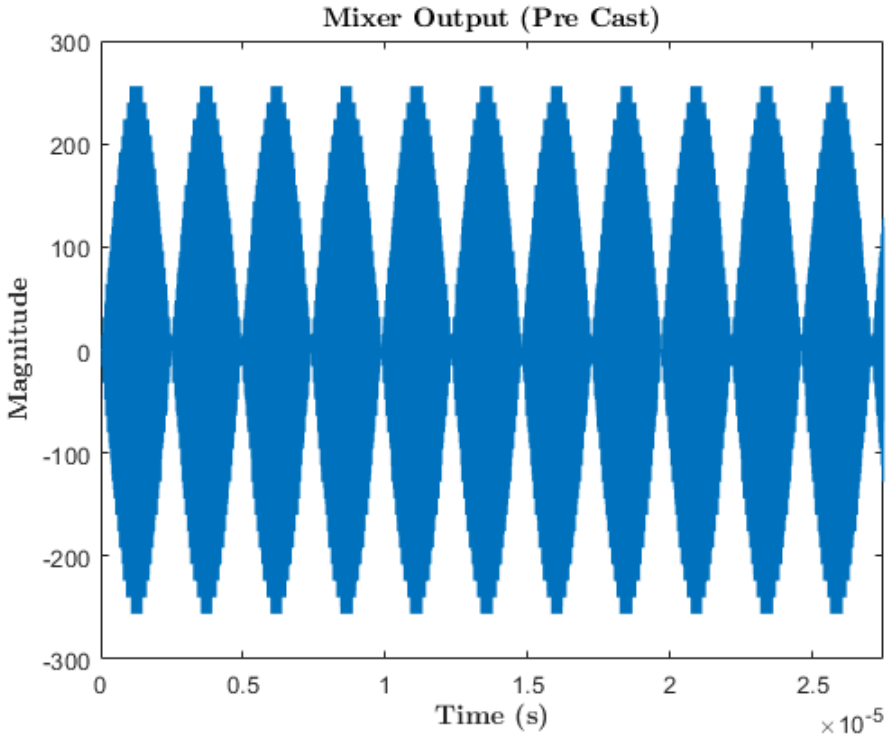

Figure 18: Transmitted Signal (Pre Cast Block)

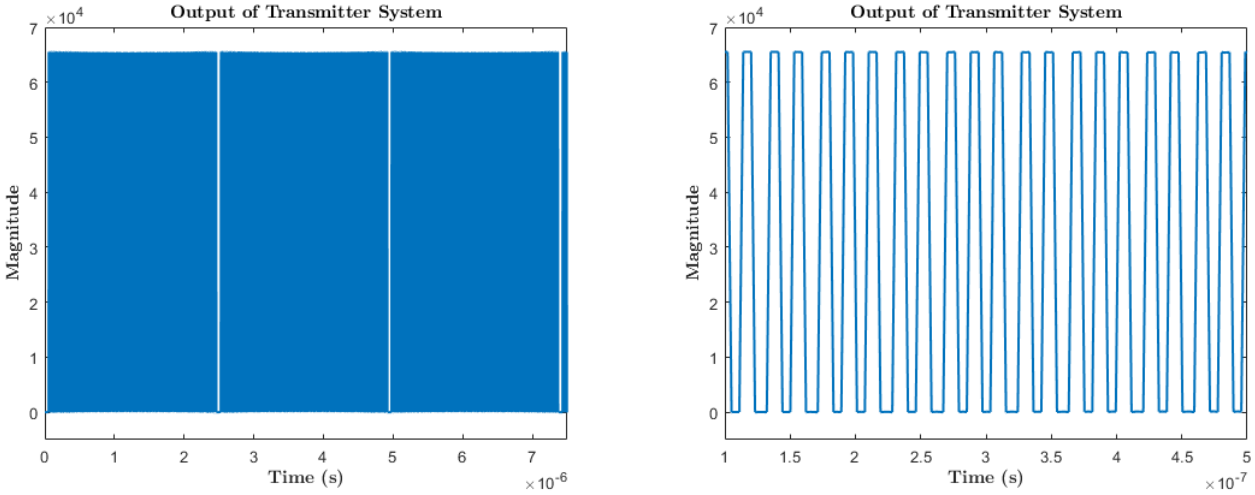

Figure 19: Transmitted Signal

signal confirms the successful AM stage, where the frequency components are now placed at $f_c - f_d$ and $f_c + f_d$, as shown in Equation 5. The frequency domain plot of this signal can be seen in Figure 20.

4.1.2 Receiver Design

When testing the receiver design, the system was initially evaluated using an ideal AM signal comprised of the multiplication of two native Simulink blocks. However, to connect a native Simulink block to a Model

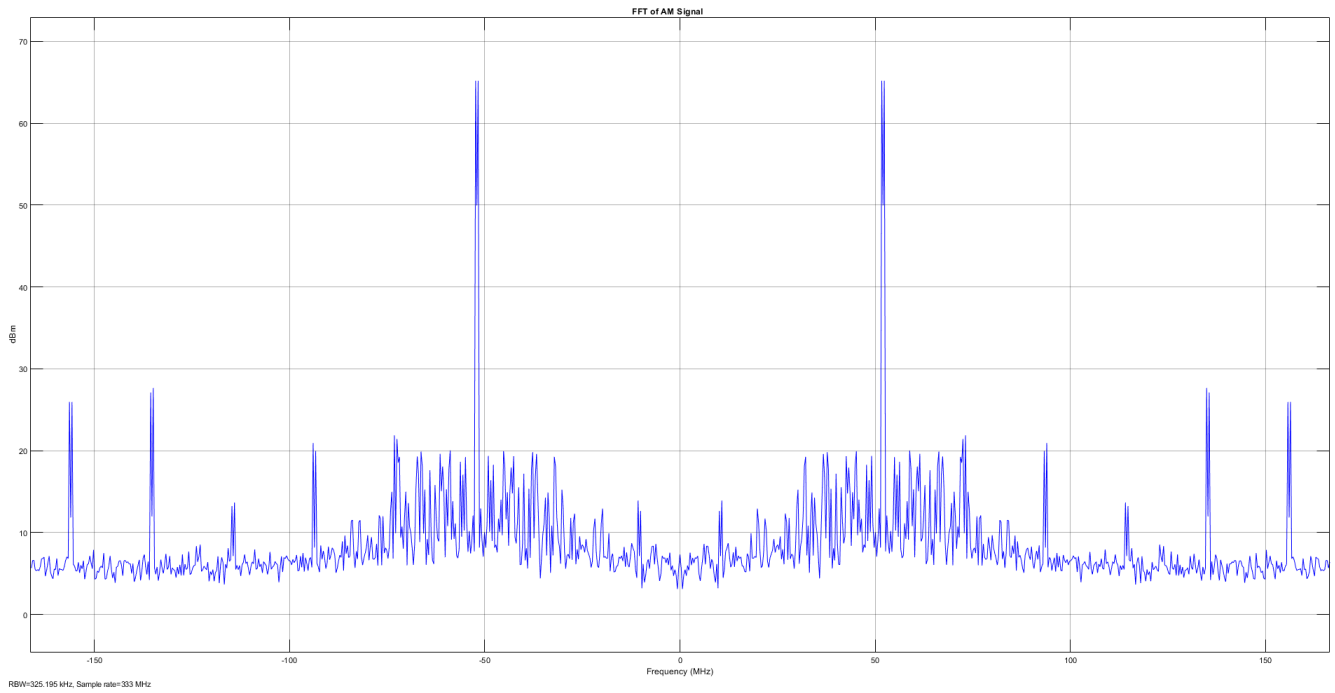

Figure 20: FFT of AM signal

Composer block, a **Gateway In** must be used to convert the Simulink continuous data into the discrete data that digital circuits use. The extra Gateway In converts the continuous data to 16-bit unsigned digital data. The output of the receiver with the ideal input can be seen in Figure 21.

4.1.2.1 Full System Testing

Along with the ideal system test, the receiver was also tested alongside the transmitter to ensure the recoverability of the transmitted signal. To do so, both Simulink models were converted into subsystems and placed within the same file. The output of the transmitter was connected directly to the input of the receiver, so the exact output could be recorded for comparison purposes alongside the hardware testing. Figure 22 shows the received signal within the full system test in the time domain.

The frequency domain plot of this signal can be seen in Figure 23.

Although this signal does exactly match the ideal output of the receiver, it does resemble the output of the transmitter. The differences in this can be down to the data width signals in the receiver system. As the FIR Compiler outputs 39 bit data, as mentioned previously, the signal is losing 7 bits of data after passing through the cast block, which can be a significant loss of data. The frequency domain plot of this signal shows the main frequency component at baseband (centred around zero), representing the transmitted frequency component.

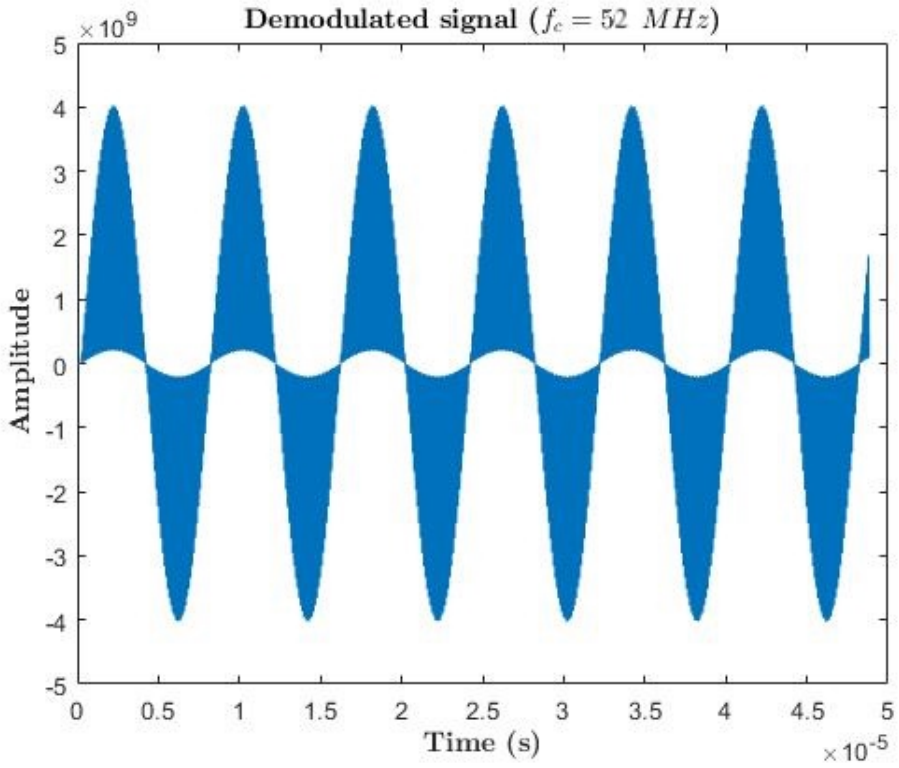

Figure 21: Demodulated Test signal

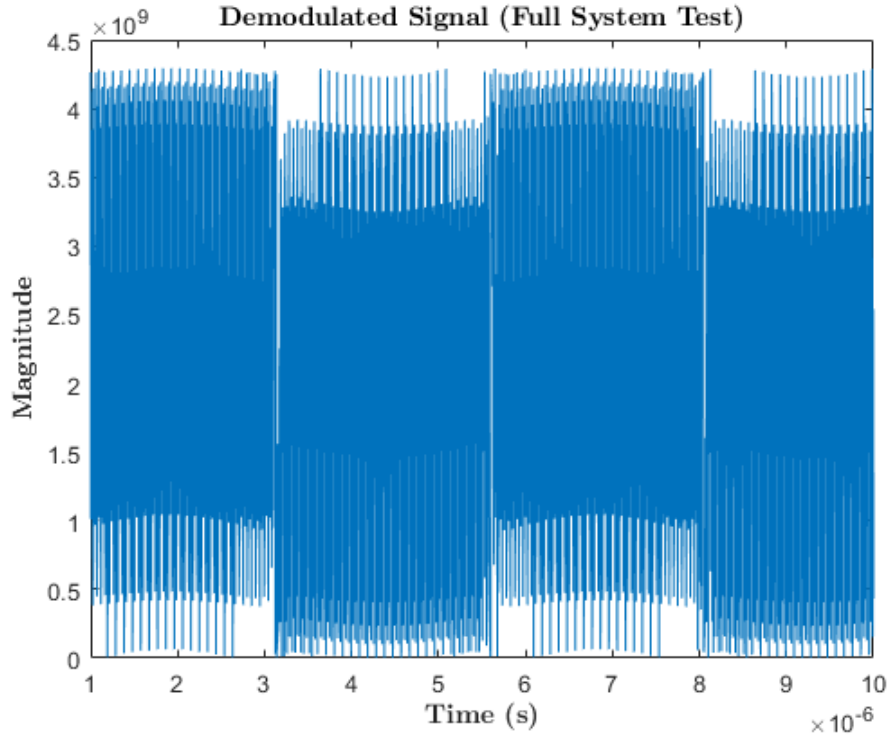

Figure 22: Demodulated Signal (Full System Test)

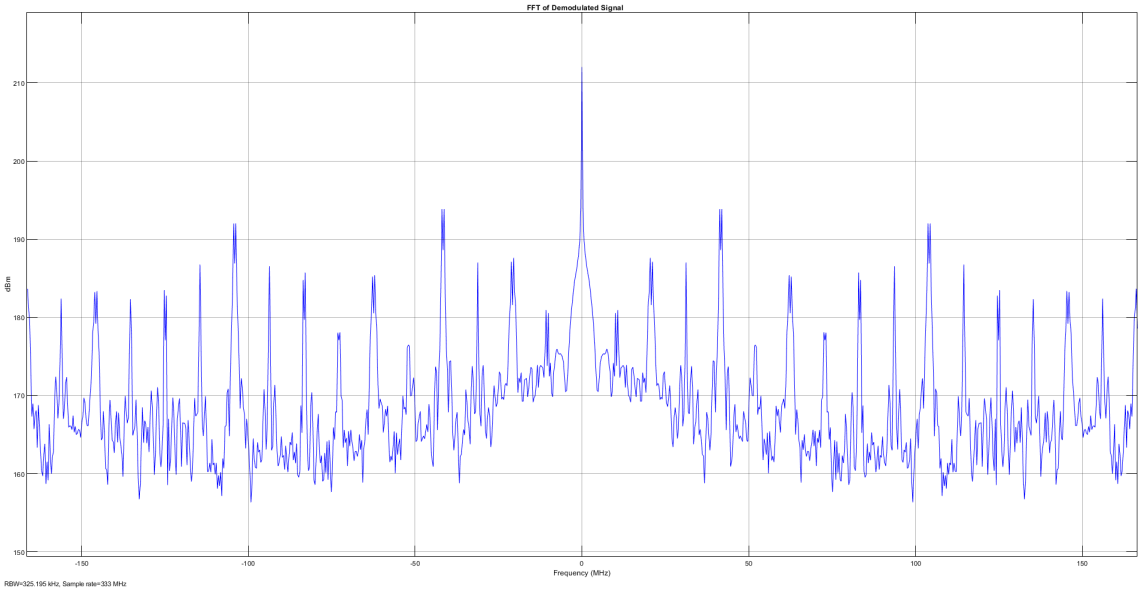

Figure 23: Frequency Domain Plot of Received Signal (Full System Test)

4.2 RFX-8440

The testing of the RFX was a significantly longer and more involved process, as the verification of hardware designs can prove a rigorous and time-consuming process. Initially, the hardware design was going to be identical to that of the 2x2, with the 2x2 primarily being used as a design verification platform, since it was more readily available than the RFX. As time progressed, however, it was realised that the RFX implementation would differ from the 2x2, as the differences in the PS interface proved a significant challenge. One of the significant challenges of the hardware integration was correctly adding the IP to the reference design. The design has a complicated folder structure, different to that of standard Vivado projects, which introduced some difficulty when instantiating the custom IP within the project. This difficulty introduced a **black box** error, where the design would not pass the implementation stage. The design was not able to see the correct IP folder during the implementation stage, so the IP could not be properly instantiated and therefore the design failed. The error is called a **black box** because Vivado cannot determine the correct inputs and outputs of the IP and therefore the design fails. To mitigate this error, the entire design was rebuilt and the most up to date IP was added to the project.

Upon the resolution of this error, the reference design could successfully pass the implementation stage, and then be loaded onto the card. As mentioned previously, this reference design is very large, using most of the resources available on the FPGA. The implementation results from Vivado can give vital information on the FPGA usage, along with timing requirements and power usage. The chip utilisation for the 2x100GbE reference design can be seen in Figure 24.

The highlighted sections in blue indicate the utilised area of the FPGA. The rectangle in the bottom left corner denotes the PS on the FPGA, which is always added along with the PS IP in IPI. Although the implemented design graph is useful for visualising the usage of the FPGA, Vivado also generates a chip

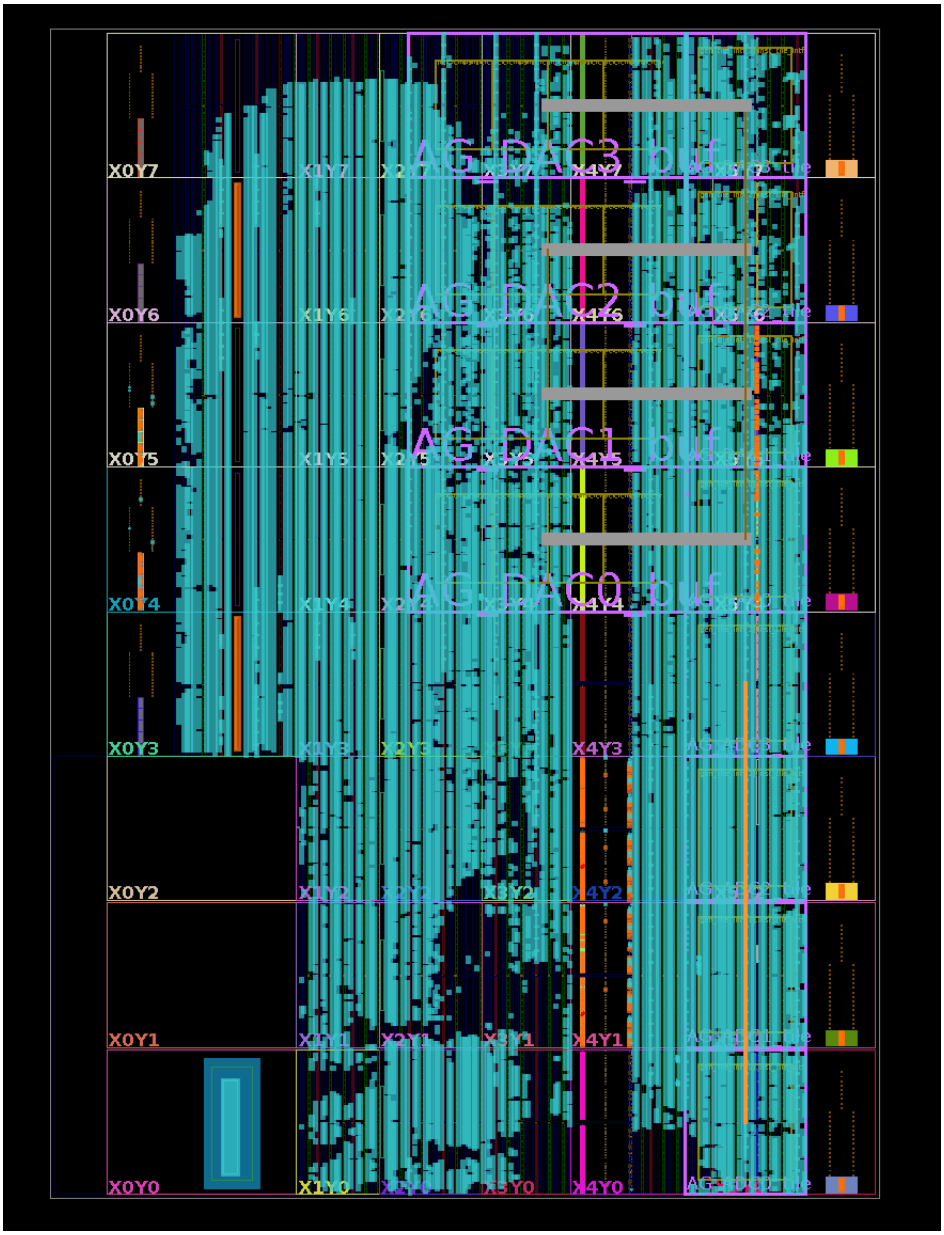

Figure 24: RFSoC Utilisation for the 2x100GbE BittWare Reference Design

usage table and graph, so the exact number of utilised FPGA building blocks can be expected. Table 4 shows the utilisation of the BittWare 2x100GbE reference design.

As previously stated, the 2x100GbE design utilises a substantial portion of the FPGAs resources. However, the majority of this utilisation is attributed to hardened blocks specific to the RFSoC, such as GTY transceivers and RFDCs. For example, this design utilises all of the GTY transceivers, along with over 90% of the available I/O on the board. However, a significant number of DSP blocks remain available, allowing for customisation. Users can leverage these dedicated DSP resources to perform additional signal processing as needed.

The Integrated Logic Analyser (ILA) IP core in Vivado is a great tool for debugging hardware, as the ILA

Resource	Utilisation	Available	Utilisation %
LUT	109843	425280	25.83
LUTRAM	12318	213600	5.77
FF	216077	850560	25.40
BRAM	598.50	1080	55.42
URAM	8	80	10.00
DSP	7	4272	0.16
IO	136	150	90.67
GT	8	8	100.00
MMCM	4	8	50.00
PLL	2	16	12.50

Table 4: BittWare 2x100GbE Reference design Resource Utilisation Summary

allows for real time logic analysis on implemented hardware, rather than relying on simulations through VHDL "testbenches". The ILA IP core is built within the project, but does not use up any resources on the FPGA, but rather stores the signals, samples, ports and data inside of the on-chip Block Random Access Memory (BRAM) [27]. The ILA connects to both the output of the transmitter block, along with the input the RFDC. Once the image is loaded onto the device, the Vivado **hardware manager** can be opened, where the card will be auto-detected, and the available ILA outputs will appear inside of the GUI. The ILA automatically detects the clock associated with the signal that is connected to it, however, if multiple clocks are connected, then multiple ILA IP cores may need to be added to avoid any clock domain crossing. Note that the ILA IP core can be added to a project in two ways. The first method is through Vivado's IPI environment, where the ILA IP core can be directly integrated, similar to other proprietary IPs. Alternatively, the ILA can be added after the design has been synthesised by selecting the **debug** option, allowing signals to be dragged and dropped into the debug wizard.

Once the PetaLinux image was successfully loaded onto the card, the minicom terminal was inspected to view the boot process of the card, ensuring the RFX was properly configured before running any tests. The first test utilised the BittWare **rfxutils** package which included a number of python scripts to run on the board. The primary test included the use of the `analyser.py` script, which generates data from the PS, then transmits and receives signals through the RFDCs using a loopback cable. This test is particularly useful because it confirms the correct configuration settings of the RFDCs, whilst also using the `ii_dsp_top.vhd` file as a bypass for the transmitted and received data. Once the data is received, the script then plots the FFT of this signal using the **numpy** Python package, which is commonly used for implementing mathematical operations in Python. Figure 25 shows the spectrum plot of the received data through the `analyser.py` script.

The spectrum analyser tool also provides options to select the desired channel, adjust transmission frequencies, and disable the transmit side. This allows another signal to be received and analysed on the spectrum. After successfully running the spectrum analyser, the next step in hardware verification involved using the `devmem` Linux command. This command is a powerful tool for modifying memory addresses in embed-

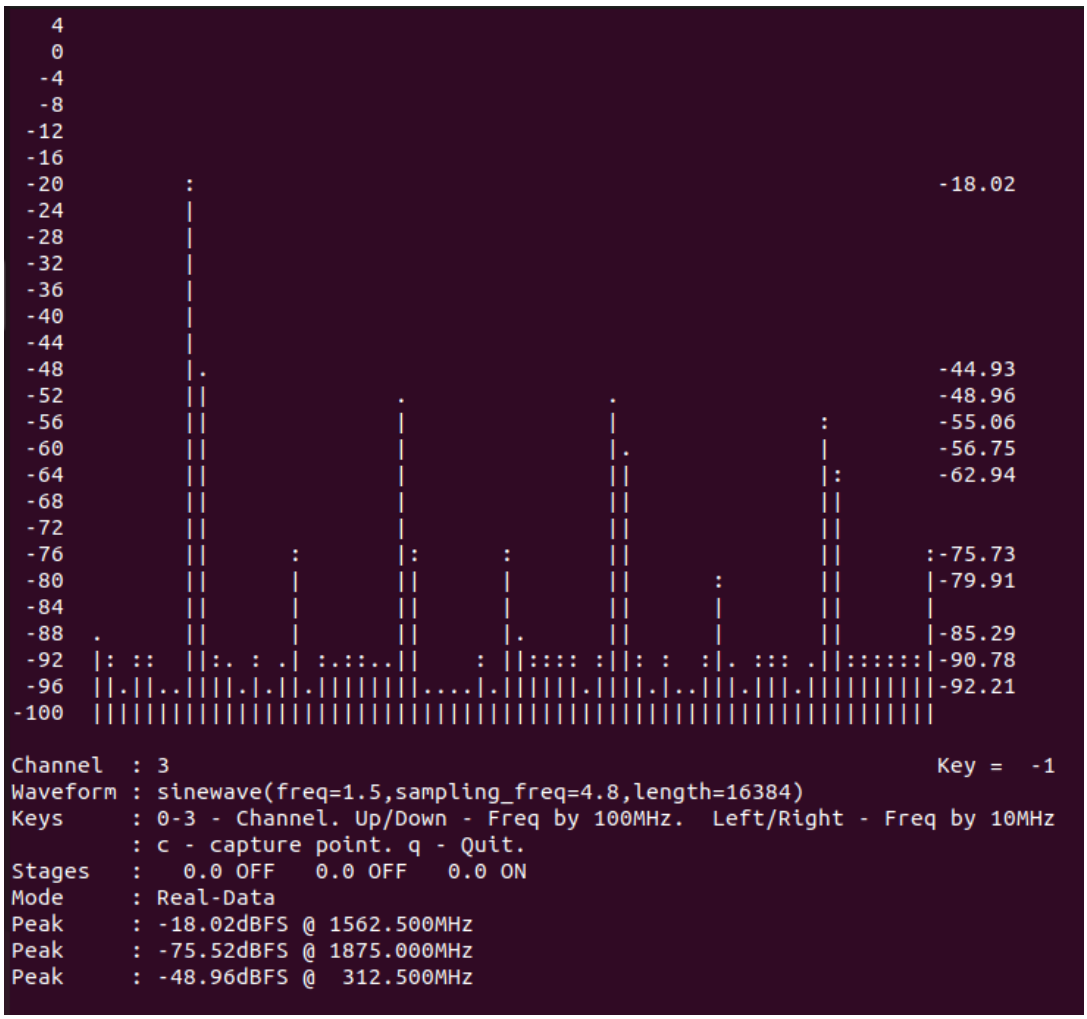

Figure 25: BittWare Spectrum Analyser Test

ded systems. Within the Vivado project for the 2x100GbE design, an address editor provides access to all memory addresses, allowing them to be inspected and modified. This is crucial for verifying user registers, which were added to interface with the transmit component within PetaLinux. Table 5 shows the memory addresses for both the MUX enable and NCO driver register.

Register	Memory Address (HEX)
NCO Driver	0xA0090008
MUX Enable	0xA009000C

Table 5: User Defined Register Memory Addresses

Each memory address is 32 bits (4 bytes), meaning consecutive addresses are spaced 4 bytes apart. As a result, the two registers are not adjacent within the memory map. Traditionally, memory addresses are represented in hexadecimal (HEX), a base-16 number system. This representation is preferred because it provides a more compact and readable format for handling large amounts of data, when compared to binary. The `devmem` command can both read and write to memory addresses, meaning the initial value of both registers could be confirmed. Listing 6 shows the syntax for writing to one of the memory addresses

within the design.

```
devmem 0xA009000C 32 0x1
```

Listing 6: Devmem Memory Address Write Operation

This command sets the MUX enable register to high by writing a **one**, which would allow the NCOs inside of the transmit component to be able to receive an input. To verify the functionality of the transmitter component, the hardware manager was opened to use the built in ILA IP cores to inspect the logic. For the ILA to know which signals need to be monitored when running the logic, the `debug_nets.ltx` file needs to be added to the hardware manager. This file is created by Vivado when the ILA is added to the design. Once the ILA was set up and ready to monitor the logic functions, the analyser test script was once again ran, this time to ensure that the RFDCs are correctly configured. With the script running and the RFDCs enabled, the MUX enable register was set to one and the NCO driver register was set to 0; this enabled the NCOs to start generating the modulated data which could then be seen inside of the ILA. The ILA visualises the data through a traditional logic graph, so only the discrete values of the waveform can be seen. To properly visualise the waveform being output from the ILA, the data was exported to MATLAB for plotting. Figure 26 shows the first waveform viewed from the ILA in both the time and frequency domain.

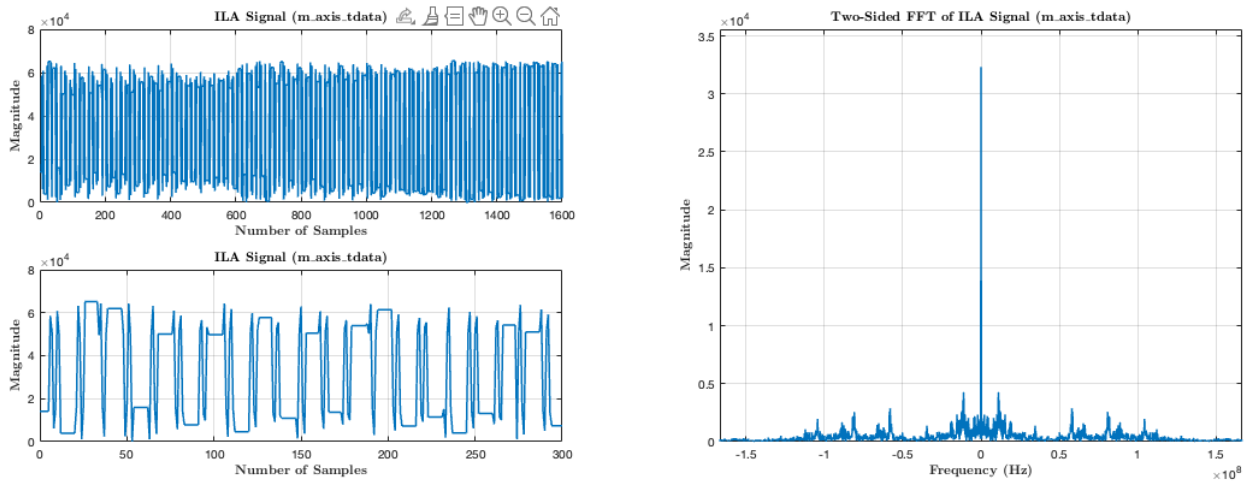

Figure 26: Time and Frequency Domain Representations of First Tx Signal Viewed in ILA

From inspection of these graphs, it is difficult to say whether the signal is properly represented here, since the signal is not particularly clear in either the time or frequency domain. It was possible that the signal was significantly undersampled, since the sampling frequency of the Model Composer system was significantly higher than the ILA clock at this point in the design and testing process. However, the sampling frequency of the Simulink system is not considered once the design has been implemented on hardware, rather the IP will take whatever clock signal is given and use this as its sampling frequency. Nevertheless, the transmitter model was revised to implement a lower sampling frequency and in turn a lower transmitted

signal frequency, which is the revision displayed within the Methodology of this report. Figure 27 shows the resulting output of this model revision when viewed on the ILA.

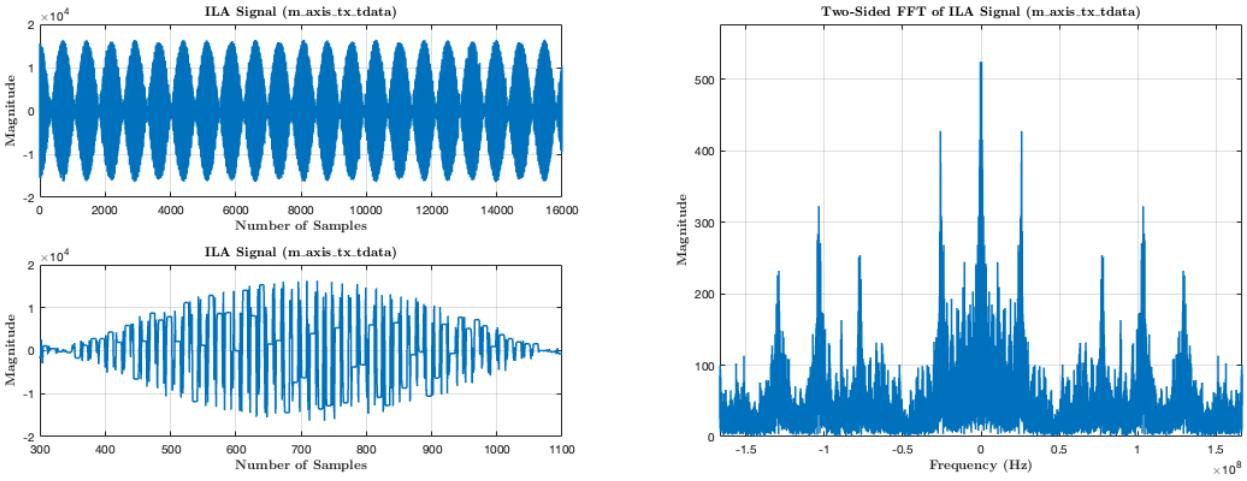

Figure 27: Time and Frequency Domain Representations of Second Tx Signal Viewed in ILA

Inspection of this figure shows a properly amplitude modulated signal. However, the signals frequency domain representation does not exactly meet the expected output, shown earlier in the simulation results. The time domain representation also does not match the expected output, as the transmitted does not perfectly output an AM sinusoid. After updating the model for the second ILA test, the ILA was measured at the input to the RF-DAC (`m_axis_tx_tdata`). Measuring the ILA debug core at this point may change the characteristics of the waveform, since the RF-DAC likely introduces a further modulation stage to transmit at significantly higher frequencies. Another possibility for the differences in this result could be due to the spectrum analyser tool, since this was running throughout the ILA measurements, the generated signal from the PS may have also been connected to the same signal as the transmit component, disturbing its generated signal. Although the spectrum analyser was running throughout this test, the signal in which the transmitter was connected to was ensured to be connected to this component when writing the VHDL component instantiations and definitions. Unfortunately, only the transmitter component was successfully implemented within the RFX design during the project's time frame. Integrating custom components into the RFX hierarchy proved to be a significant challenge and was considerably more complex than the RFSoC 2x2 design flow. However, despite the RFX's limited functionality, the experience gained was highly valuable. Working with an enterprise-level device and gaining hands-on experience in a professional engineering environment provided invaluable insights for both professional and academic development.

4.3 RFSoC 2x2

The RFSoC 2x2 presented a distinct challenge compared to the RFX, requiring a fully customised hardware design. This complexity introduced several new difficulties, offering valuable opportunities for learning and advancement in both hardware and software development.

4.3.1 Hardware Implementation

The hardware design for the 2x2 was much simpler than that of the RFX, but still presented challenges when trying to implement a loopback set-up in hardware. Within the hardware design, one of the most significant challenges faced was the clocking. As mentioned earlier, there are a number of different clock domains within the design, with all needing to be properly managed to avoid any clock domain crossing, leading to failed design implementations. One of the most significant challenges presented by the 2x2 was the configuration and management of different clock domains. As mentioned earlier, when dealing with the RFDC IP core, the clocking configuration for this device produces two different clock domains, even if the RF-DAC and RF-ADC output frequencies are the same. With improper clock configuration, the design may still pass the implementation stage, allowing for the bitstream generation to occur. However, in practice, if the RFSoC 2x2 does not receive proper reference clocks, then the board may crash and communication will be cut, leading to a significant amount of debugging. The RFDCs need to receive reference clocks from the board to then output their required clocking value. This requires the correct board files to be added into the Vivado project, which differs from the RFX, as this is just built based on the chip. After adding the correct board files, an external port must be added to interface with the reference clocks on the board. This port is named `lmk_reset` and is set as a constant zero. This allows for the LMK clock to be set and resolves the crashing issue of the board in hardware. This is a board specific issue to the 2x2, and is not present in more modern boards, so finding this issue took significant effort and involved building a different RFSoC 2x2 design and inspecting any differences within the block diagram. Along with the addition of the external port is the new constraints file that needs to be added, which tells Vivado to connect certain signals inside of the hardware design to different physical output pins on the FPGA. This is also the case for the RFX design, however, the constraints file will need to be significantly larger to accommodate for the mapping of pins to a custom board. Constraints are defined within Xilinx Design Constraints (.xdc) files, using the Tcl language. Listing 7 displays the contents of the constraints file.

```
set_property PACKAGE_PIN AT9 [get_ports "lmk_reset[0]"]
set_property IOSTANDARD LVCMOS18 [get_ports "lmk_reset[0]"]

set_property PACKAGE_PIN AP9 [get_ports reset]
set_property IOSTANDARD LVCMOS18 [get_ports reset]
```

Listing 7: Contents of `constraints.xdc` File for RFSoC 2x2 IPI Design

The addition of the `lmk_reset` fixed a significant design problem, which was causing the board to crash in software whenever any of the IP was initialised through the DMA. Since the custom transmit and receive IP were clocked at the same rate as their respective RFDC side, the communication between the the PS and these IP could not be established if the RFDC does not receive the correct reference clock. Similarly to the RFX, the implemented design in hardware showcases import resource utilisation characteristics of the board, and where different resources are allocated. Figure 28 shows the resource utilisation of the RFSoC

2x2 design.

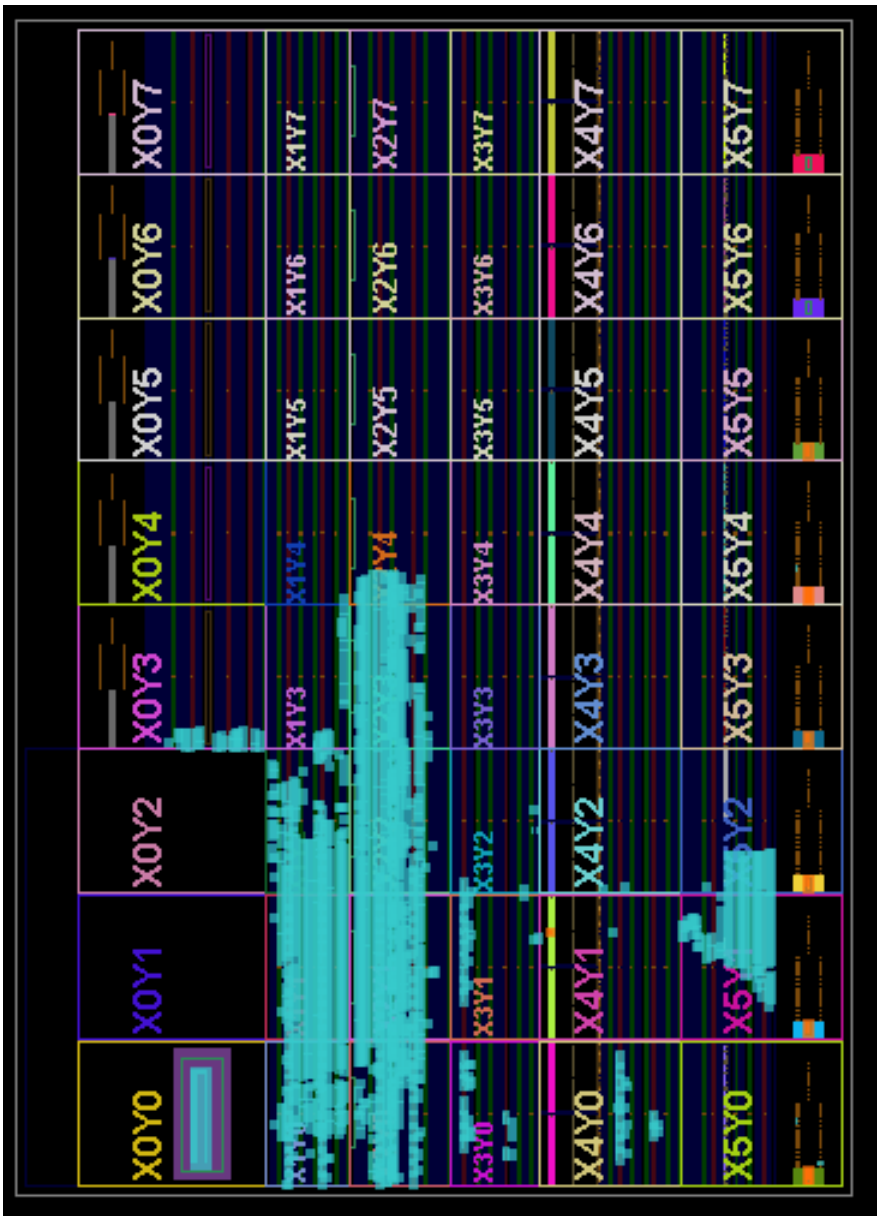

Figure 28: RFSoC Utilisation for RFSoC 2x2 Design

Comparing Figures 24 and 28 shows that the RFSoC 2x2 design uses significantly less resources on the FPGA than the 2x100GbE design, since it implements far less logic and only uses one of the available RFDC channels. Again, the best way to understand the different allocation of resources on the 2x2 is through the table which Vivado produces alongside this graph. Table 6 shows the resource allocation for the RFSoC 2x2 design.

The ILA also played a crucial component in the testing and debugging of the hardware design, with ILA debug cores added to the inputs and outputs of both DMAs to ensure proper AXI4 handshakes were taking place, along with data being received and produced from the DMAs read and write channels.

Resource	Utilisation	Available	Utilisation %
LUT	18120	425280	4.26
LUTRAM	7804	213600	3.65
FF	23448	850560	2.76
BRAM	145	1080	13.43
DSP	103	4272	2.41
IO	1	347	0.29
MMCM	1	8	12.50

Table 6: RFSoC 2x2 Resource Utilisation Summary

4.3.2 PYNQ

Testing the 2x2 design was significantly faster compared to the RFX, as PYNQ can run any FPGA bitstream without requiring a complete OS image rebuild. This reduces the development cycle's lengthy processes to just bitstream generation, which takes approximately an hour. However, if a design causes the board to crash, as was the case here, any modifications require over an hour before the design can be verified again. As mentioned previously, to transmit and receive data, the DMA is used to allocate and manage the memory needed for these transactions. To send a DMA buffer, the `sendchannel` is used. This channel takes data from the PS and sends it to the input of the transmitter IP, which generates the signal. This signal is then sent to the RFDC and back into the receiver IP. From there, the DMA buffer is then read using the `recvchannel` at the output of the receiver. Unfortunately, each DMA inside of the transmit and receive hierarchies are set up to only allow write and read operations, respectively, so the transmitted signal cannot be viewed from the Jupyter notebook using PYNQ. Figure 29 shows the received signal of an earlier revision of the transmitter and receiver models.

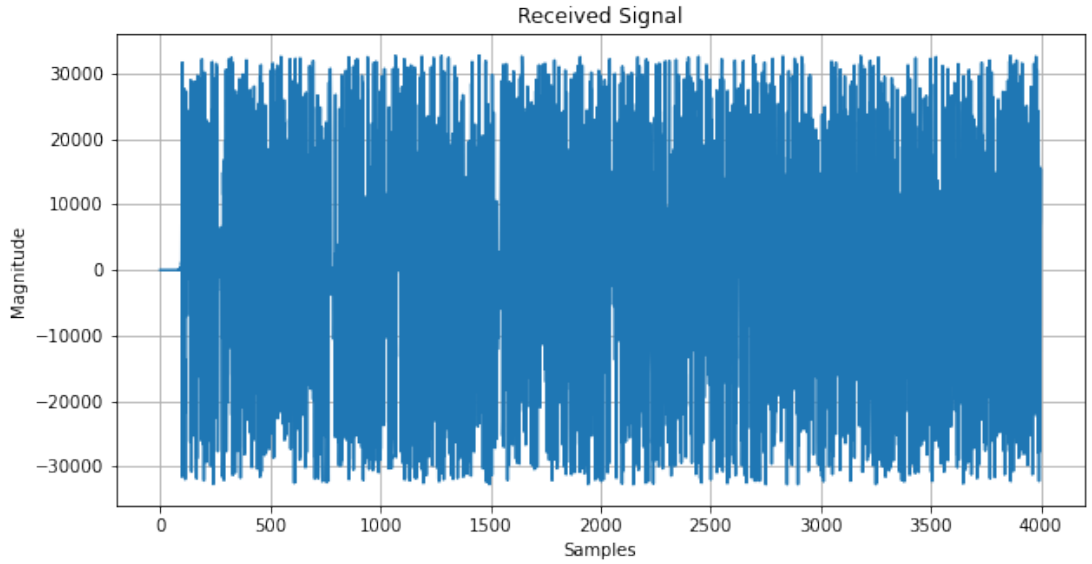

Figure 29: PYNQ Received Signal - Early Revision

As mention previously, the signal was plotted using the matplotlib package, so the figures look slightly different when compared to the simulated results from Model Composer. Figure 30 shows the FFT of this received signal.

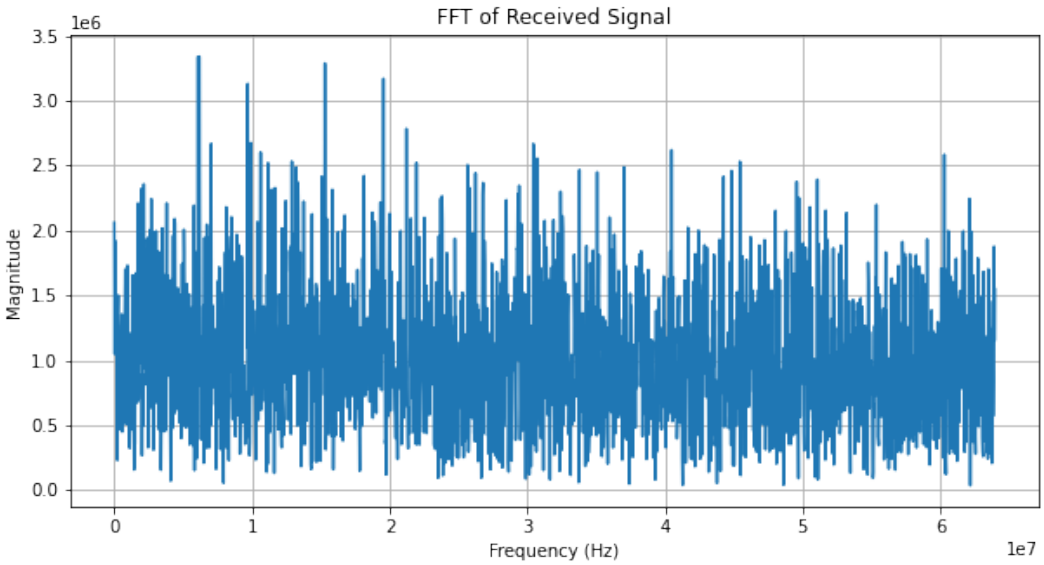

Figure 30: FFT of Received Signal - Early Revision

During the initial testing of this revision of the design, this resulting output was thought to have been an issue, since it does not resemble the expected output as seen from the Model Composer simulations. However, this resulting output is actually "correct", as the simulations confirmed that the transmitter was outputting a similar signal to what is seen above. The same model was used for the PYNQ design and the RFX, meaning that the expected clock was 333 MHz to interface with the RFX design properly. However, the clock that was given to both the transmitter and receiver IP was set to 128 MHz, meaning that the modulating frequency and carrier frequency were set to 38.45 kHz and 20 MHz, respectively. Inspection of a Figure 30 shows a significant frequency component at the carrier frequency, however, there are other frequency components which don't match the expected frequency spectrum.

These are some of the reasons the system was revised, to try and output a clearer signal which could be more easily recognised during testing. After revising the models to the version in which they are presented within this report, the resulting received signal can be seen in Figure 31.

This figure shows a more expected output at the receiver, however the signal is still not as clear as the desired AM signal, but does still match the expected output from the simulated results. Figure 32 shows this FFT of this signal.

The FFT of this signal produces a more expected output, showing a dominant frequency component centred around zero, along with additional frequencies near the carrier frequency at approximately 20 MHz. However, some discrepancies may be present in the plot due to the DMA buffer containing only 4000 samples, which significantly reduces the FFT resolution compared to the frequency plot from the simulations.

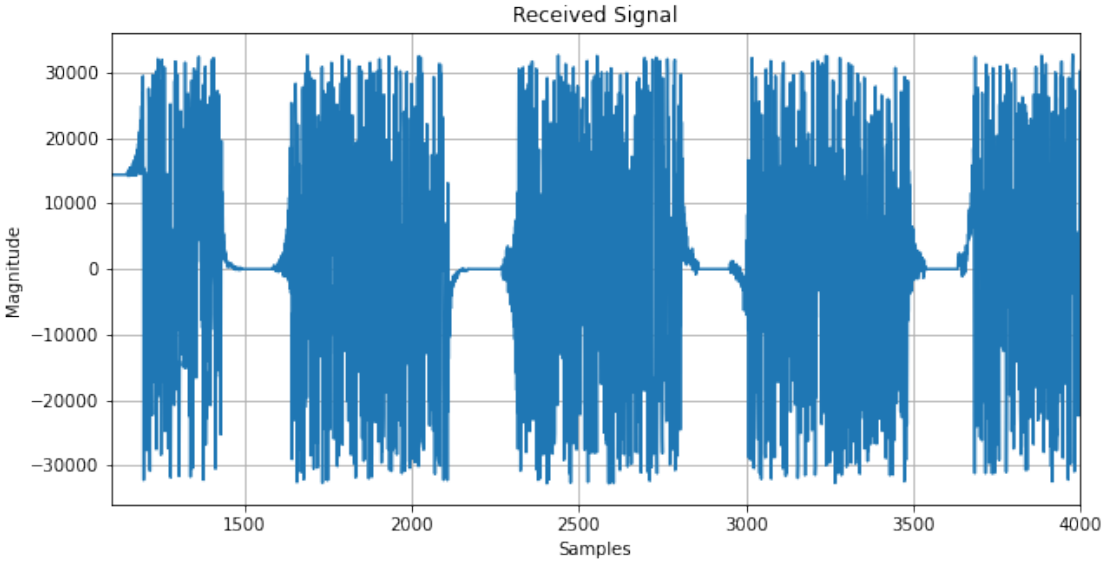

Figure 31: PYNQ Received Signal - Later Revision

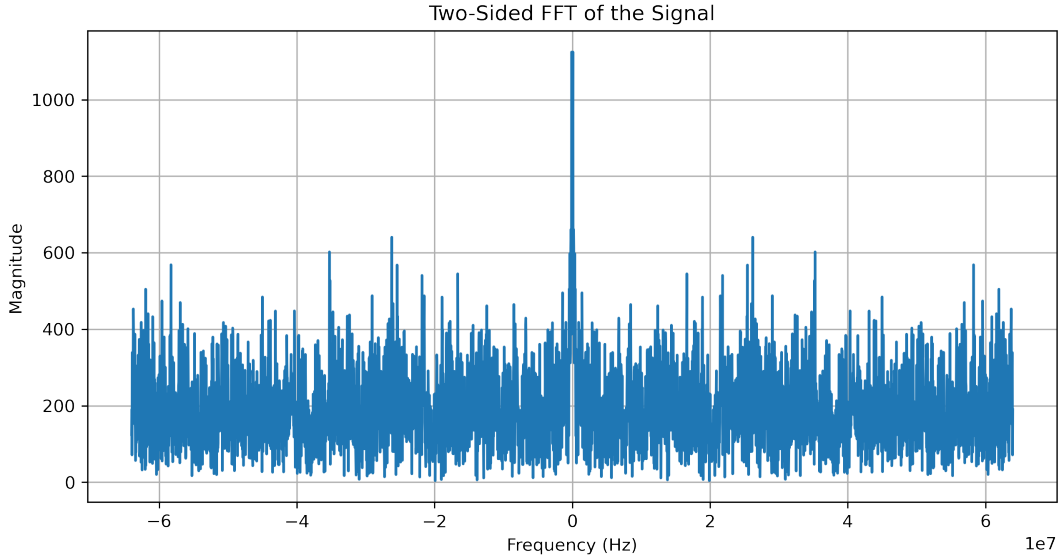

Figure 32: FFT of Received Signal - Later Revision

A comparison between the results from the later revision of the model and its corresponding simulation reveals a strong similarity in both the time and frequency domains. Although the transmitter does not perfectly generate the desired AM signal, the received signal still aligns with expectations, with the expected frequency components appearing in the FFT plot. Although the transmitted waveform does not perfectly match the desired AM signal, the system is still capable of receiving and processing the same waveform it transmits, even if this is not ideal. A significant portion of the project development was dedicated to interfacing designs within the RFX platform, with the primary focus being the PetaLinux image. Initially, the plan was to develop a fully custom PetaLinux image, however, this focus later shifted toward integrating

custom IP within the 2x100GbE design, which has a far more streamlined build process, as the designs have been tested and verified before.

5 Conclusion and Further Work

5.1 Conclusion

In conclusion, this project provided valuable experience in SDR and embedded systems development, including their development cycles, software, and workflows. The process involved a significant learning phase, as both FPGA programming and SDR are deeply rooted in historical and mathematical principles, requiring a strong foundation in DSP, communication theory, and digital electronics.

Various software tools were utilised throughout the project for SDR development, providing a robust platform for understanding the key objectives required to design and implement a functional SDR system on the RFSoC platform.

Overall, the project was largely successful, resulting in the development of an SDR application within the BittWare RFX platform. AMD's Vitis Model Composer was extensively used for designing the digital logic of the transmitter and receiver systems. This tool offers a layer of abstraction over the traditional, labour-intensive method of writing RTL code by providing pre-defined blocks that represent different hardware components. Both the transmitter and receiver were designed using this software and subsequently exported as custom IP cores for integration within the Vivado IPI environment.

GNU Radio was used as a tool for learning and understanding the architecture of different communication techniques. Although GNU Radio was not implemented within the design, it still proved very useful in aiding in the early phases of the project, before the focus shifted to more hardware-focused objectives.

A significant portion of the development effort was dedicated to understanding the RFX platform, which differed from previously used FPGA devices due to its unfamiliar PS interface and reliance on PetaLinux. Building PetaLinux images is a time-consuming process, with builds taking over an hour, leading to an extended design cycle, especially in contrast to the RFSoC 2x2 and PYNQ platforms.

Although considerable time was spent on the RFX and its operation, its implemented design had significantly limited functionality compared to the RFSoC 2x2. This was primarily due to the restricted availability of the RFX, as it could only be accessed at the BittWare office, typically twice per week. As a result, designs could not be tested as frequently or as rigorously as on the 2x2, especially given the lengthy development process for both PetaLinux and the 2x100GbE reference design, which consumed a significant portion of the available time in the office. Despite these limitations, the experience gained from working with an enterprise-level device was invaluable. Such exposure is uncommon in undergraduate projects and provided valuable insight into professional engineering practices.

The RFSoC 2x2 platform had a far more successful and functional SDR application than the RFX, since this board was available to be used during university opening hours, so the development time could be allocated to the 2x2 when the RFX was not available. The 2x2 design successfully allowed for the transmission and reception of a generated signal, although the signal was not in the desired format, it still matched the

expected output from simulations.

5.2 Further Work

In future, further functionality could be added to the RFX design, as only the data generation block was able to be implemented within the limited time frame in which the RFX was available.

The transmitter and receiver designs could be upgraded to feature a more complex and realistic modulation schemes. This would allow for the system to be useful within more real-world SDR systems, with proper source encoding and timing/phase synchronisation if transmitting across a non-ideal channel.

Integrating GNU Radio within the system would provide a robust and modular software design, as different visualisation/receiver architectures could be implemented through a QSFP offload system. This system would send packeted data from the RFX to a host PC running GNU Radio, which can then implement any sort of DSP necessary.

References

[1] T. F. Collins, *Software-Defined Radio for Engineers*. Norwood, Massachusetts; London, England: Artech House, 2018. Accessed: Jan. 3, 2025.

[2] GNU Radio, "What Is GNU Radio." `https://wiki.gnuradio.org/index.php?title=What_Is_GNU_Radio`. Accessed: Jan. 6, 2025.

[3] AMD, "AMD Zynq™ UltraScale+™ RFSoC." `https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-ultrascale-plus-rfsoc.html`. Accessed: Jan. 6, 2025.

[4] L. H. Crockett, D. Northcote, and R. W. Stewart, eds., *Software Defined Radio With Zynq UltraScale+ RFSoC, First Edition*. Strathclyde Academic Media, 2023. `https://www.RFSoCbook.com`.

[5] Advanced Micro Devices (AMD), "Amd adaptive support article 1053914: Axi basics 1 - introduction to axi," 2023. Accessed: 2025-03-02.

[6] D. Northcote, "EE315: Further VHDL and FPGA Design - AXI Protocols and Direct Memory Access," 2024. Accessed: March 2nd 2025.

[7] "PYNQ: Python Productivity for Zynq," 2025. Accessed: 2025-02-03.

[8] AMD, "AMD Vitis™ HLS." `https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html`, Nov 2024. Accessed: Dec. 03, 2024.

[9] AMD, "AMD Vitis™ Model Composer." `https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-model-composer.html`, Nov 2024. Accessed: Dec. 03, 2024.

[10] BittWare, "Rfx-8440l l-band direct sampling transceiver." `https://www.bittware.com/products/rfx-8440l/`, Oct 2024. Accessed: 2024-12-03.

[11] A. Marwanto, M. A. Sarijari, N. Fisal, S. K. S. Yusof, and R. A. Rashid, "Experimental study of ofdm implementation utilizing gnu radio and usrp - sdr," in *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, pp. 132–135, 2009.

[12] C. V. Năstase, A. Marţian, C. Vlădeanu, and I. Marghescu, "Spectrum sensing based on energy detection algorithms using gnu radio and usrp for cognitive radio," in *2018 International Conference on Communications (COMM)*, pp. 381–384, 2018.

[13] M. Šiaučiulis, L. H. Crockett, and R. W. Stewart, "Ultra-wideband SDR architecture for AMD RFSoCs and PYNQ based GNU Radio blocks," *Proceedings of the GNU Radio Conference*, vol. 8, no. 1, pp. 1–8, 2023. Accessed: October 27th 2024.

[14] S. Weiss, I. Tavakkolnia, "EE320 Signals and Communications Systems Part 1: Signals & Systems," 2023/24. Accessed: 20th Feb 2025.

[15] University of Strathclyde - Software Defined Radio Research Laboratory, "pynq_sobel." https://github.com/strath-sdr/pynq_sobel#, 2024.

[16] L. H. Crockett, "EE315 Slides - Numerically Controlled Oscillators," 2023. Accessed: Feb 23rd 2025.

[17] J. R. Magers, "Criteria for maximum spurious free dynamic range of a receiver system," in *2013 European Microwave Conference*, pp. 1515–1518, 2013.

[18] Advanced Micro Devices, Inc. (AMD), "AMD University Program - RFSoC 2x2." https://www.amd.com/en/corporate/university-program/aup-boards/rfsoc2x2.html, 2025. Accessed: March 28th 2025.

[19] L. Harvie, "Understanding and Applying Clock Domain Crossing Techniques in FPGAs," *Runtime Rec*, 2024. Accessed: March 11th 2025.

[20] Xilinx, "RFSoC 2x2 Board Files." https://xilinx.github.io/RFSoC2x2-PYNQ/board_files.html. Accessed March 2nd 2025.

[21] BittWare, a Molex Company, "RFX-8440 Framework Logic Guide." https://www.bittware.com/products/rfx-8440a/, 2024. Accessed: 19th March 2025.

[22] Advanced Micro Devices, Inc. (AMD), "Tcl Scripting in Vivado." https://docs.amd.com/r/en-US/ug894-vivado-tcl-scripting, 2024. Accessed: March 9th 2025.

[23] B. Chandhoke, "AND8459/D - Basics of Clock Jitter." https://www.mouser.com/pdfdocs/src-tutorials/Basics-of-Clock-Jitter.pdf?srsltid=AfmBOopigZfPtEAaZ21DE9MVU0lZaZeXcYXjptvD_IKCb21V0SsJuXQg. Accessed: 15th March 2025.

[24] Advanced Micro Devices, Inc. (AMD), "Zynq 7000 SoC Software Developers Guide (UG821)." https://docs.amd.com/r/en-US/ug821-zynq-7000-swdev/U-Boot. Accessed: March 26th 2025.

[25] J. V. Hook, "What is jtag? exploring the fundamentals." Americas, https://www.zuken.com/us/blog/what-is-jtag-exploring-the-fundamentals/, mar 2024. Accessed: 1st March 2025.

[26] Noridc Semiconductors, "QSPI — Quad serial peripheral interface." https://docs.nordicsemi.com/bundle/ps_nrf52840/page/qspi.html. Accessed: March 20th 2025.

[27] Xilinx, "Integrated Logic Analyser v6.2 - LogiCORE IP Product Guide." https://docs.amd.com/v/u/en-US/pg172-ila, 2016. Accessed: March 27th 2025.

Appendix A Project GitHub Repository

The projects supplementary material has been provided through the Myplace, however the GitHub repository can be viewed online for ease of viewing.

The GitHub repository can be found here: `https://github.com/fraseroregan77/FO-RFSoC-4YP.git`

To help understand the flow of the repository and simplifying what files may be where, the following guide can be used.

Folder	Description
2x2_files	Folder containing all Vivado projects for the RFSoC 2x2 development.
GNURADIO	Folder containing all GNU Radio development and examples.
Overlays	Folder containing all Overlays used throughout the development **PYNQ_Design_wrapper** is the working model.
PYNQ	All files used for PYNQ - specifically Jupyter notebooks.
RFX_files	Folder containing all files used for RFX development. namely the model composer designs (used for both RFX and 2x2).
images	All images needed for report writing and results.
report	Backed-up files for report (Tex files)

Table 7: Description of GitHub Repository